

MSSACT-Net 遥感土地覆盖分割

项目报告

报告编号: 项目报告_20260913 **数据集:** LoveDA 公开基准 (0.3m 航空正射, 7 类土地覆盖, 官方人工标注) **模型:** MSSACT-Net (多尺度自注意力 ConvTransformer, light 6.10M / full 30.54M) **报告日期:** 2026-09-13

数据来源声明: 本报告全部数字由项目仓库的实验记录自动汇总生成
(`FACTS.json` $\leftarrow$ `checkpoints/*_history.json` + 评估结果 JSON), 可逐项追溯, 不包含手工填写或估算值。

摘要

本项目在公开遥感基准 **LoveDA** 上系统训练与评估 **MSSACT-Net** (多尺度自注意力 ConvTransformer), 并完成与 7 个主流分割模型的对比、8 个模块消融、多组训练策略探索、以及针对**标签质量上限**的专项验证。主要结论:

- 数据量与质量共同决定性能上限。** 训练数据由 957 张扩至 1,768 张 (并剔除 前 10% 高 no-data 的低质量图、修复标签映射缺陷) 后, 主模型 Kappa 由 **0.4065** 提升至 **0.6312**, OA 提升至 **0.7030**。
- 全部五个设计模块在数据充足时均无可测量的独立贡献。** 8 个消融变体的 Kappa 落在 0.627–0.637, 与完整模型 (0.6312) 的最大差异仅 **0.006**; 其中 **5 个变体 (含替换可学习解码器的 Bilinear-Up)** 反而略优于完整模型。FPN 与 Transformer 合计占 67% 的参数, 其移除对性能无显著影响。
- 收敛速度主要由学习率调度决定, 而非架构。** 通过累计学习量 (LR 积分) 分析: 12 个模型中 9 个达到 Kappa 0.5 所需的累计学习量集中在 47%–64%, 说明"第几轮收敛"是调度曲线的映射, 不是架构优势。**唯一例外是 DeepLabV3+ (31.8%)**, 其空洞卷积 + ASPP 多尺度结构确实具备更高的学习效率。

4. 标签质量已成为模型能力评价的瓶颈。 6 个架构/容量差异极大的模型（2.4M–39.6M）逐像素分析显示：模型间一致率 **0.7589** 显著高于模型与人工标签的一致率 **0.6495**；多模型共识与人工标签仅 **0.6815** 一致；且**容量-性能相关性 发生反转**（人工标签下 $r=+0.311$ ，共识参考下 $r=-0.444$ ）。表明标注精度已成为 评价模型能力上限的约束。

5. 方法学工作。 建立数据泄漏防控（MD5 去重 + 三集互斥校验 + 干净测试子集）、类别体系核对（依据官方文档确认「背景」与「未分类」的区别并修复标签映射缺陷）、收敛协议设计（Kappa 早停 + EMA + OneCycle）与模型间一致性分析框架。

关键词： 高分辨率遥感；土地覆盖分类；语义分割；Transformer；自注意力；LoveDA；数据量效应；标签噪声；消融实验

0. 项目状态

项	状态
数据集	LoveDA 官方训练区全量获取 (2,522 张)
数据筛选与划分	✔ no-data $\leq 10\%$ 筛选 $\rightarrow$ MD5 去重 $\rightarrow$ 8:1:1 (train 1768 / val 221 / test 221 / test_clean 141)
泄漏校验	✔ 三集两两互斥；test_clean 与历史训练集零重叠
主模型训练	✔ full_all_v2 (60 轮) + post_all_v2 (后训练 30 轮)
对比实验 (7 个基线)	✔ 完成
消融实验 (8 个变体)	✔ 完成 (散布 0.006–0.008 Kappa； 实测 $\sigma_{\text{seed}}=0.0050$ ，故不可分辨，见 5.7)
标签质量专项验证	✔ 完成 (一致性分析 + 边界分层)
训练策略探索	✔ 完成 (阶段 A 11 组含 SGDR； 阶段 C 裁剪策略见 5.8)
架构改进实验 (D1–D4)	✔ 完成 (同管线同协议；见 5.6.9)
数据量阶梯 (250/500/1000)	✔ 完成 (见 5.9)
预算攻击 (15 轮)	✔ 完成 (见 5.10)
随机种子噪声底线	✔ 完成 ($\sigma_{\text{seed}} = 0.0050$ ，见 5.7)
裁剪策略 (随机/固定、256/384/512)	✔ 完成 (见 5.8)

项	状态
DeepLabV3+ 从零训练对照	✅ 完成（推翻原 +0.0720 的架构归因，见 5.6.6）
架构缺陷数值检验 (D1/D2/D3)	✅ 完成（见 5.6.2–5.6.4）
等效性检验（TOST）	📁 材料已就绪（eval_rigor.py），待补齐
跨数据集验证、多光谱适配	📅 后续计划（见第 8 节）

1. 项目背景

1.1 研究目标

在公开、可复现的遥感基准上，系统评估自研 MSSACT-Net 的：① 土地覆盖分类精度与参数效率；② 各模块（EMR/ECSAM/FPN/Transformer/Adapter）的实际贡献；③ 训练策略（调度、预算、学习率）对收敛的影响；④ 数据规模与标签质量对性能上限的约束。

1.2 为什么改用公开数据集（早期疏漏说明）

早期研究采用自选研究区（国产高分卫星影像, 2m 分辨率）配合公开标签产品（CLCD, 30m 分辨率）。该方案存在两个根本缺陷：

- 1. **标签与影像尺度不匹配**：30m 标签重采样到 2m 影像造成标注与地物错位，监督信号不可靠
- 2. **类别分布极端**：该区域森林占比超过 80%，模型倾向单类预测即可获得较高 OA，结论不具备代表性与可比性

改用 LoveDA（0.3m 航空正射 + 官方人工逐像素标注 + 社区公认基准）后，类别分布均衡（最大类背景 45.6%）、标签精度可靠、结果可与文献直接对比。

2. 数据

2.1 官方类别体系（依据官方文档核对）

Category labels: **background – 1, building – 2, road – 3, water – 4, barren – 5, forest – 6, agriculture – 7**. And the **no-data regions were assigned 0** which should be ignored.

标签值	类别	说明
0	no-data (未标注)	官方要求忽略，不参与训练与评估
1	背景 (Background)	官方正式类别，占比最大 (约 33–38%)
2	建筑	
3	道路	
4	水域	
5	裸地	
6	森林	
7	农田	

要点：「背景」是官方正式类别，**不是**「未分类」；真正的未分类是值 0。

2.2 数据演进三阶段

阶段	数据	划分	标签处理	用途
A	957 张 (镜像子集)	单划分	⚠ 含 no-data 映射缺陷	历史结果 (第 4.1 节)，仅供参考
B	2,257 张 (官方全量去重)	8:1:1	⚠ 仍含缺陷	中间结果 (第 4.2 节)
C	1,768 张 (筛选后)	8:1:1 + test_clean	✅ 已修复	最终结果 (第 4.3 节起)

阶段 A/B 的标签映射缺陷：no-data (值0) 被错误并入「背景」类，导致约 3% 的无标注像素参与训练与评估，指标虚高约 **0.007–0.009 Kappa** (已量化，见 3.4 节)。阶段 C 已修复为官方协议 (值 0 → ignore)。

2.3 数据筛选与划分 (阶段 C)

步骤	结果
官方训练区原图	2,522 张
no-data 占比分布	75% 的图无 no-data; 312 张含 10%–53% 的 no-data
筛选 (no-data ≤10%)	剔除 312 张, 保留 2,210 张
MD5 去重	剔除与历史镜像重复的样本
8:1:1 划分 (seed=42)	train 1768 / val 221 / test 221
test_clean	141 张 (额外排除历史模型训练集, 用于跨阶段公平对比)

2.4 泄漏防控 (已校验)

- $\text{train} \cap \text{val} = \emptyset, \text{train} \cap \text{test} = \emptyset, \text{val} \cap \text{test} = \emptyset$
- $\text{test_clean} \subseteq \text{test}$
- $\text{test_clean} \cap \text{历史训练集} = \emptyset$
- 筛选后训练集 no-data 均值 0.13%、最大 8.27% (阈值内)

3. 方法与协议

3.1 模型架构

MSSACT-Net: Stem 卷积 → 4 阶段 EMR 残差编码器 (逐阶段 ECSAM 坐标注意力) → FPN 特征金字塔 → Transformer 编码器 (含 Adapter-Scale) → 转置卷积解码器。

配置	embed_dims	Transformer	参数量
light (本报告主模型)	[32, 64, 128, 256]	2 层 4 头	6.10M
full (原版)	[64, 128, 256, 512]	4 层 8 头	30.54M

3.2 训练协议 (所有实验一致)

项	设置
优化器	AdamW (lr=2e-4, weight_decay=0.02)
调度	OneCycle (6% warmup + 余弦退火, 60 轮)

项	设置
权重平滑	EMA (decay=0.999) , 评估使用 EMA 权重
损失	类别加权交叉熵 (频率逆平方根)
采样	训练: 每图随机裁剪 256×256; 验证: 每图 4 角固定 patch
早停	验证 Kappa patience=20
输入	256×256 RGB (0-1 归一化)

3.3 评价指标

- **Kappa 作为选优与早停标准**: 校正随机一致性。验证集最大类 (背景) 占 45.6%, 全预测背景的 OA=45.6% 但 Kappa=0——OA 在不平衡数据上会被单类支配。
- 辅助指标: OA、mF1、mIoU、混淆矩阵、每类 PA(召回)/UA(精度)。
- **数据使用协议**: val 仅用于早停/选优 (torch.no_grad() 内评估, 不参与梯度 更新) ; 最终结论以 test / test_clean 为准。

3.4 no-data 处理与缺陷修复 (重要)

官方协议: no-data (值 0) 必须忽略。

已修复缺陷: 早期 REMAP 实现将值 0 映射为 0 (背景类) , 导致约 3% 的 no-data 像素被错误参与训练与评估。修复后同一模型指标变化:

	OA	Kappa
修复前 (0→背景)	0.6862	0.6114
修复后 (0→ignore)	0.6770	0.6040
差异	-0.0092	-0.0074

即此前指标**虚高约 0.007-0.009**。阶段 C 的全部实验使用修复后协议。

4. 实验结果

4.1 阶段 A: 早期实验 (957 张, 含缺陷) — 历史结果

⚠ 本节数据含 no-data 映射缺陷，且训练集仅 957 张，**仅作历史记录**，不
作为结论依据。

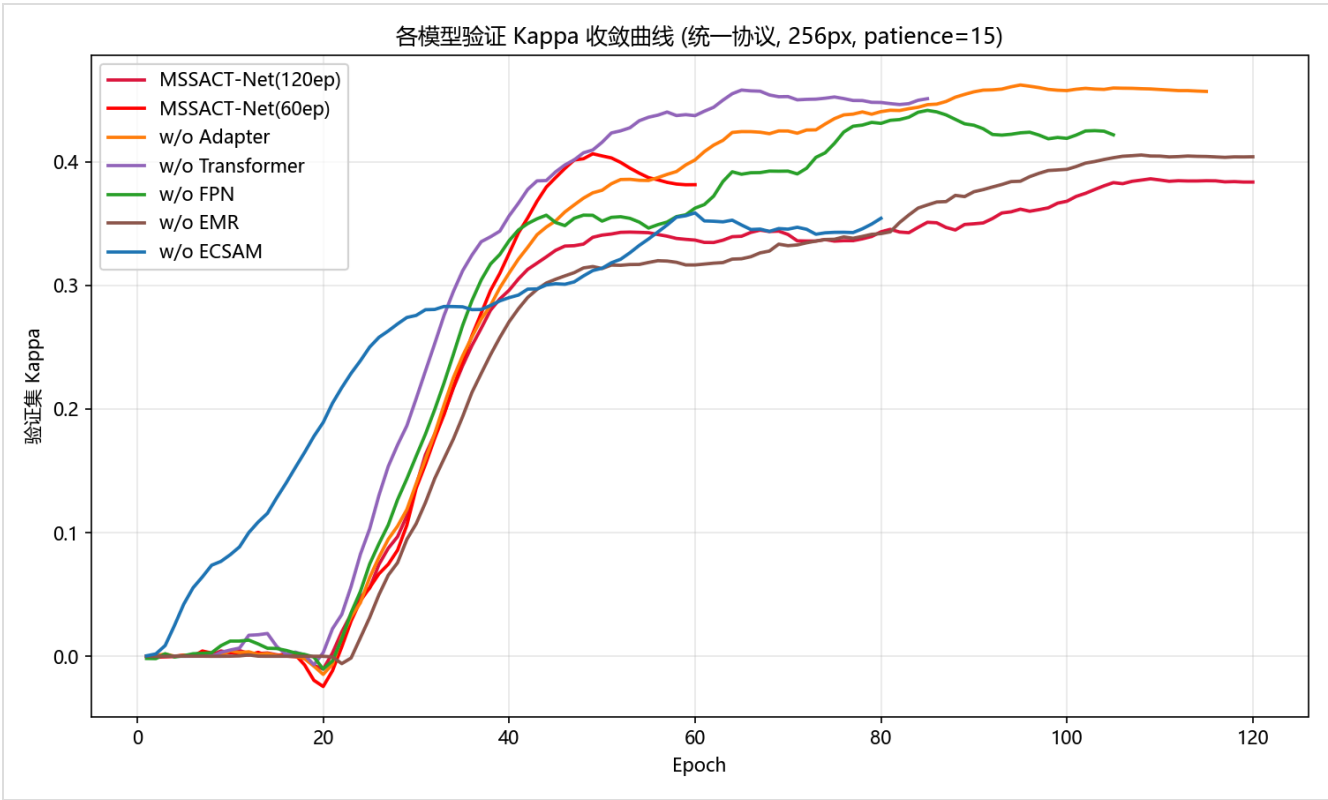
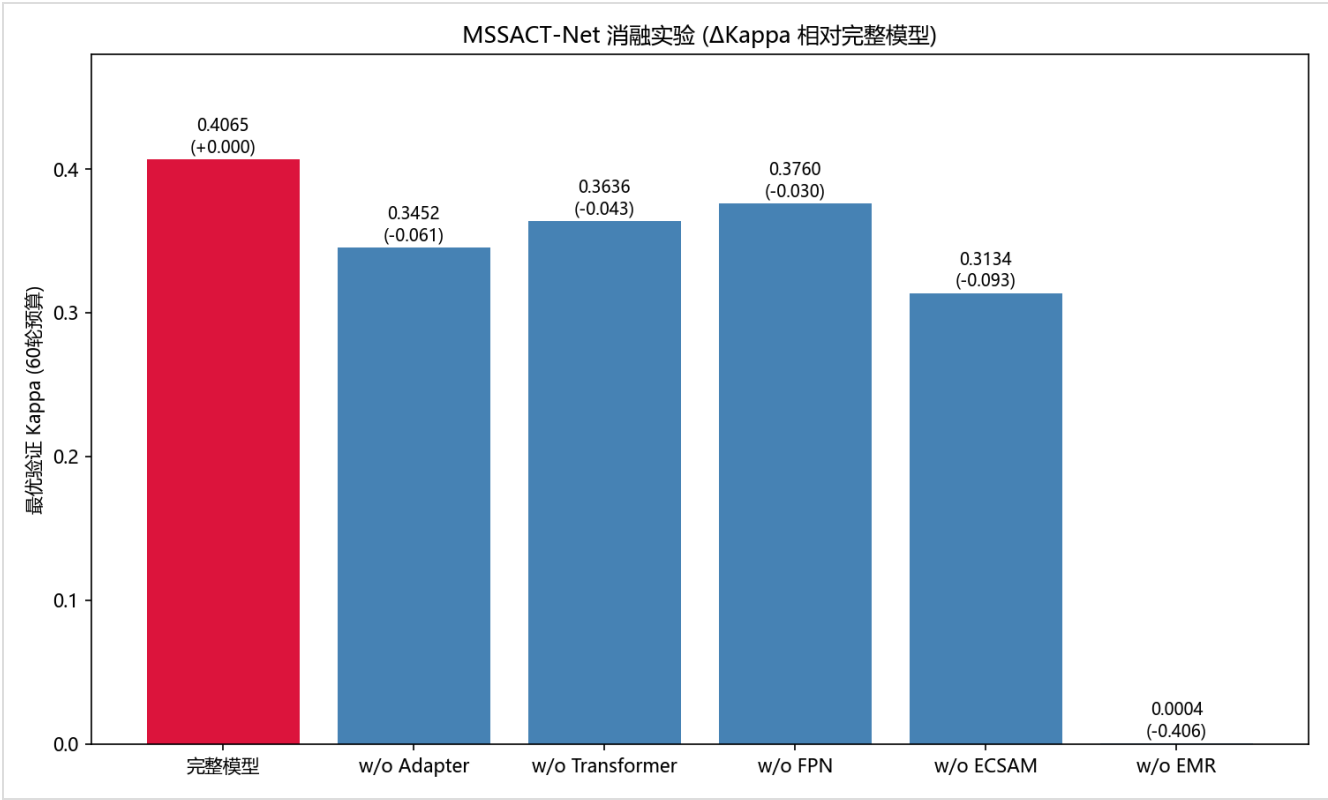
对比实验（60 轮预算）

模型	Kappa	OA	最优轮/总轮
mssact_full60	0.4065	0.5630	49/60
deeplab	0.3806	0.5092	42/57
fcn	0.2837	0.4245	35/50
pspnet	0.2513	0.3834	37/52
segformer	0.1775	0.3550	44/59
fpn_seg	0.2982	0.4364	60/60
swin_unet	0.2682	0.3991	40/55

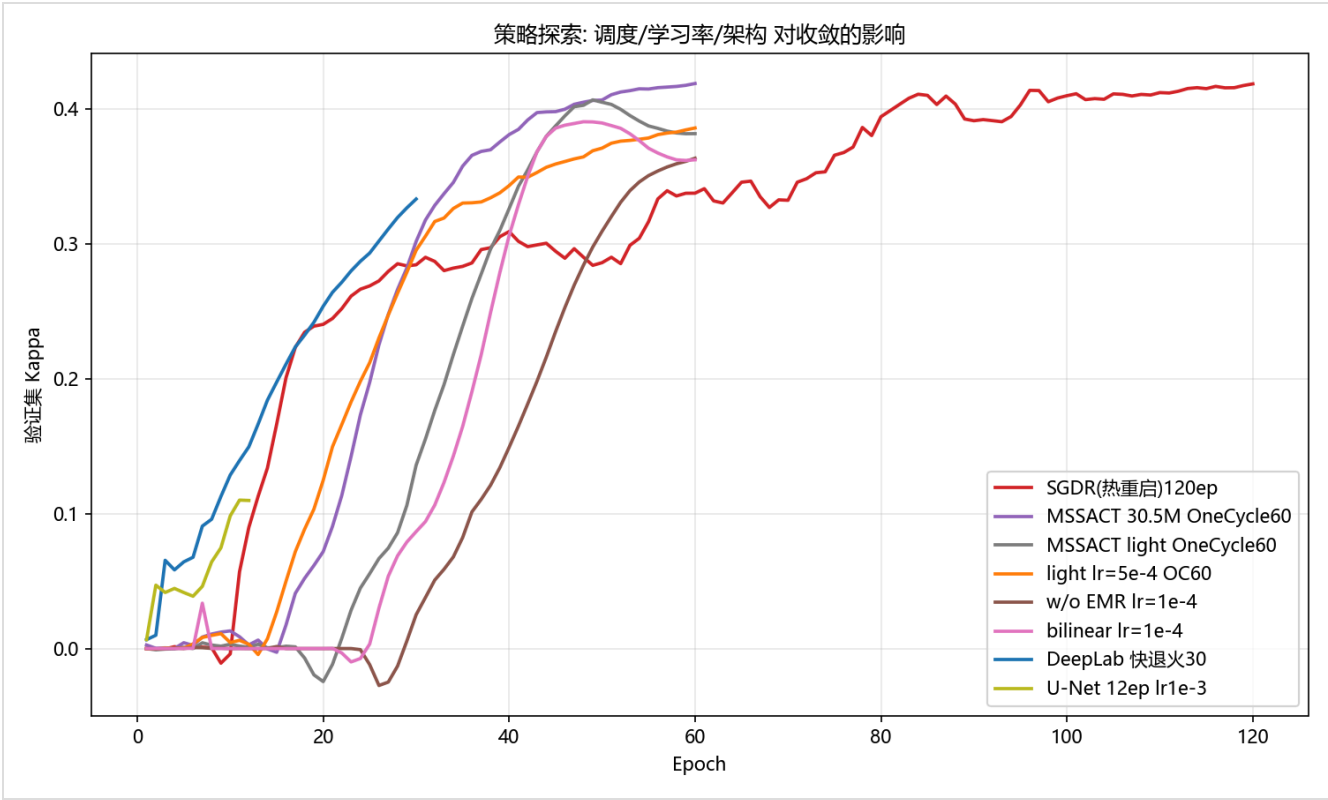
多尺度 patch 对照

模型	Kappa	OA	最优轮/总轮
v3_s256	0.3862	0.5416	109/120
v3_s512	0.2187	0.3339	16/36
v3_s1024	0.2203	0.3452	30/31
v3_s128gf	0.0098	0.3038	22/38

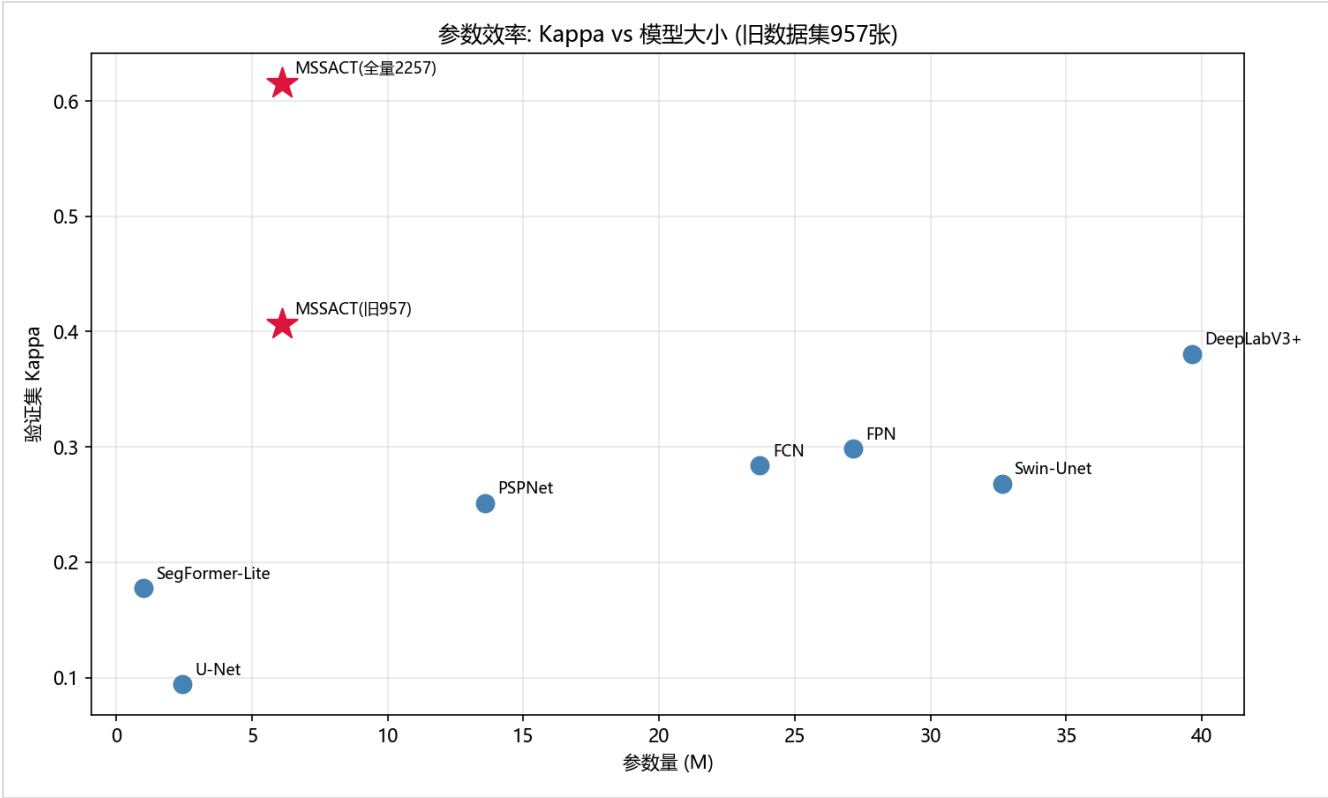
消融实验曲线（60 轮 / 120 轮预算对比）：



训练策略探索 (SGDR / 不同预算 / 不同学习率) :



参数效率 (阶段 A 数据) :



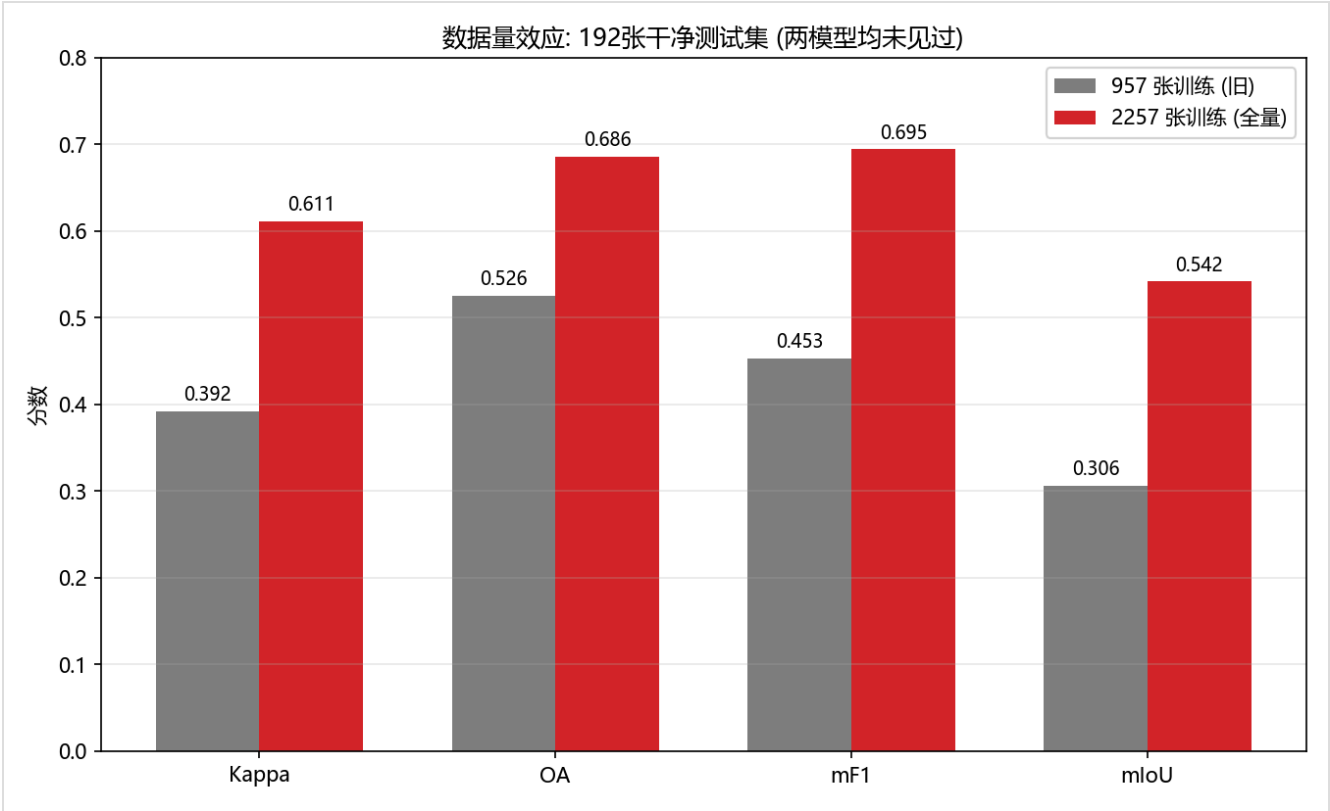
阶段 A 的主要发现（供后续阶段验证）： - 256px 在本数据量下为最优尺度 - 0.3m → 2.4m 跨分辨率直接迁移失效（Kappa≈0.01） - 训练策略探索：SGDR 热重启最优；30.5M 原版架构在调优策略下可收敛

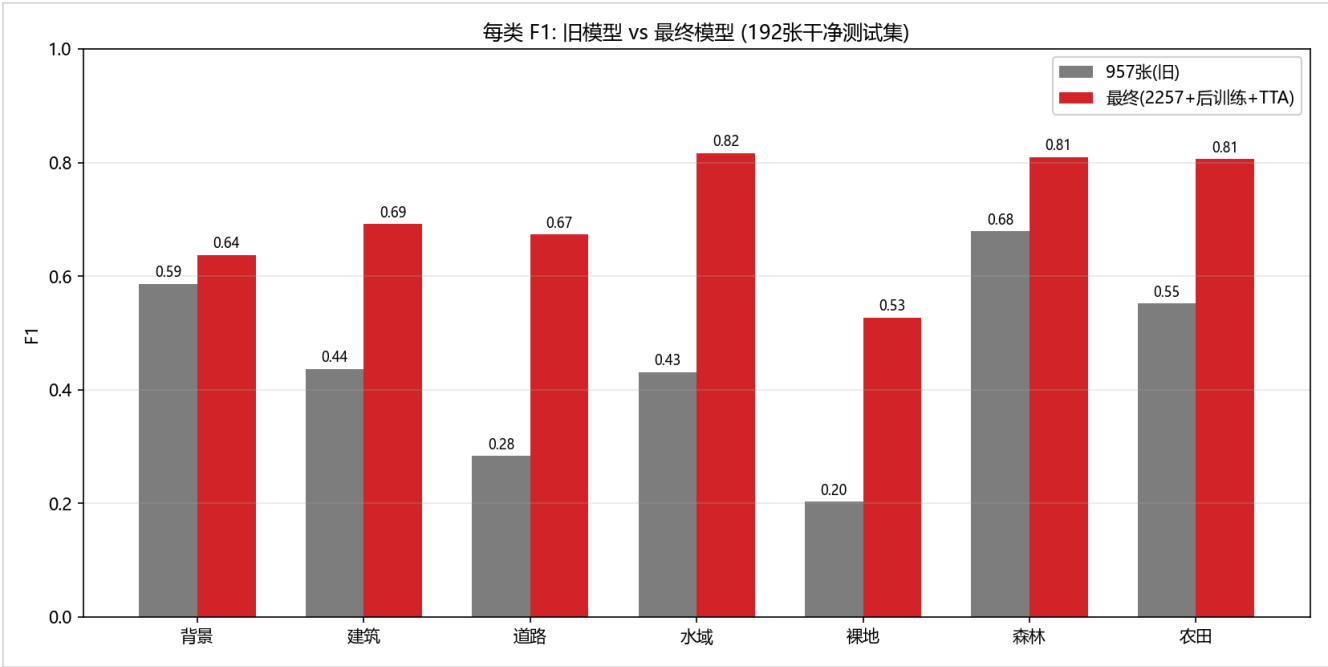
4.2 阶段 B：扩大数据（2,257 张） — 中间结果

⚠ 仍含 no-data 映射缺陷。

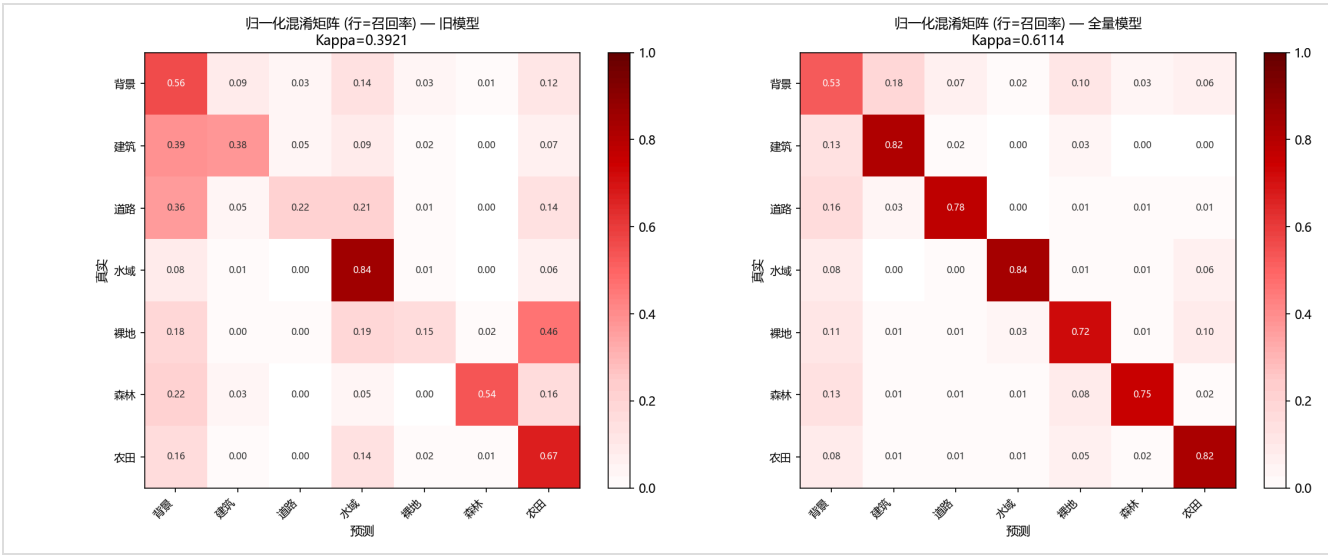
配置	Kappa	OA
full_all（主模型）	0.6155	0.6924
post_all（后训练）	0.6239	0.7003

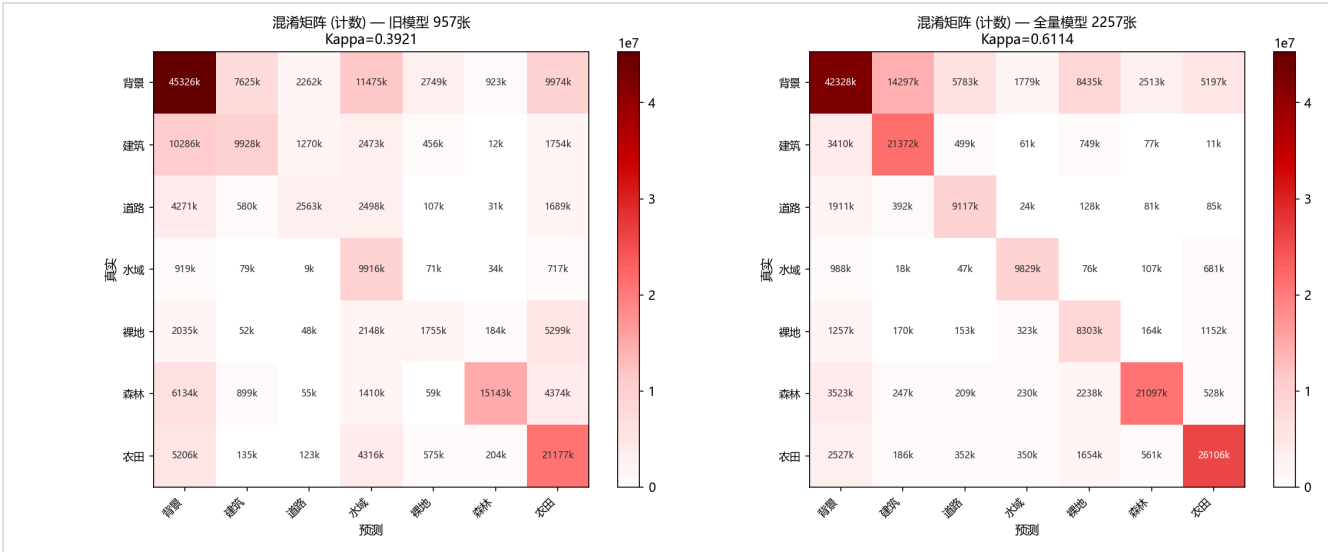
核心发现：数据量由 957 → 2,257 张，Kappa 提升约 **+0.2**，少数类（道路/水域/裸地/建筑）F1 提升最大（+0.24 ~ +0.39）——直接证明**此前的精度 瓶颈是数据量不足**。



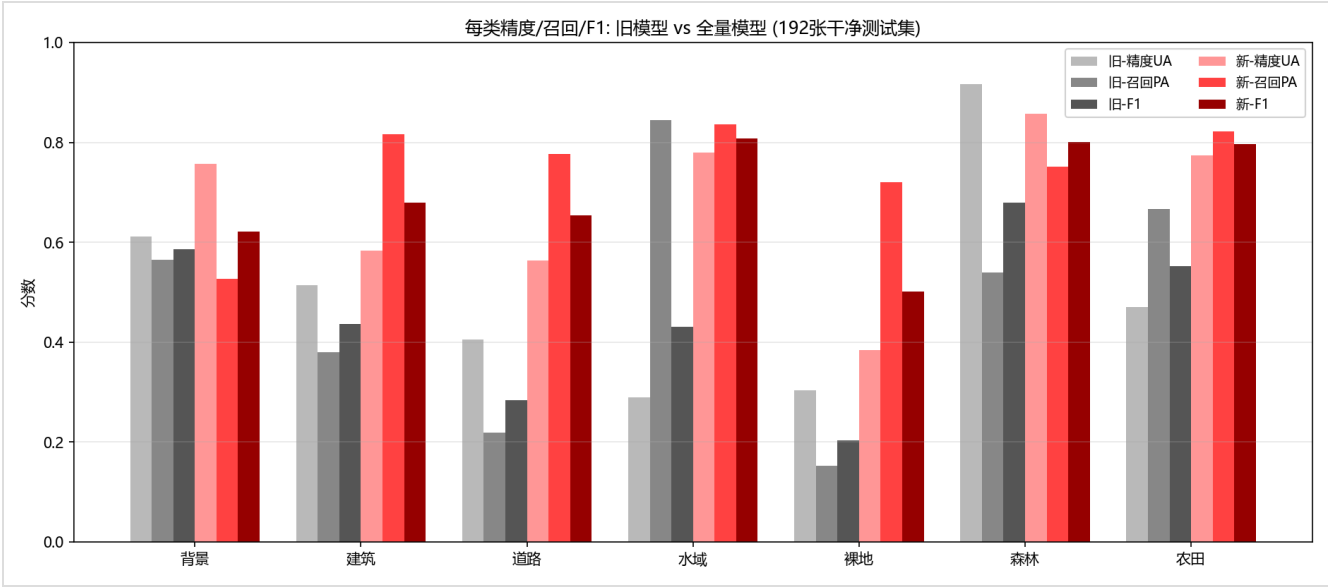


混淆矩阵 (阶段 B 主模型 vs 阶段 A 基线, 192 张干净测试集) :

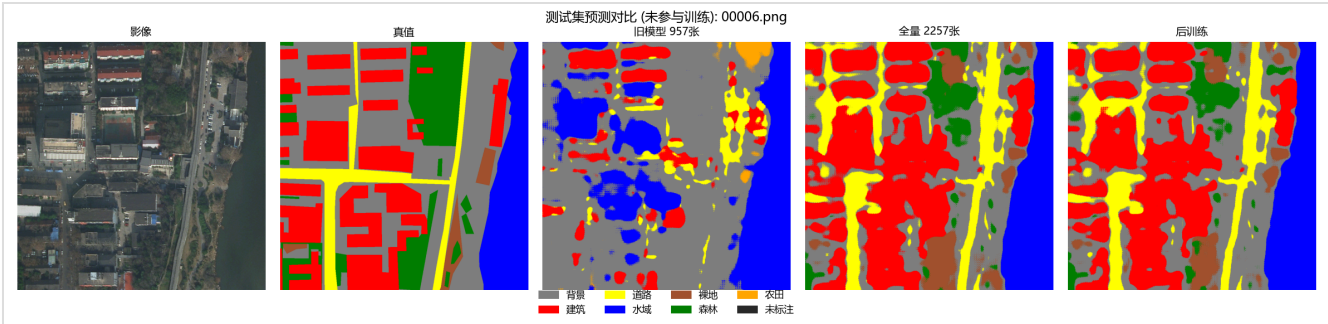




每类 精度(UA)/召回(PA)/F1：



测试集预测对比 (影像 | 真值 | 阶段 A 模型 | 阶段 B 模型)：



4.3 阶段 C：最终实验（1,768 张筛选 + 修复） — 结论依据

4.3.1 主模型与后训练

主模型（完整 MSSACT-Net light）

模型	Kappa	OA	最优轮/总轮
full_all_v2	0.6312	0.7030	54/60
post_all_v2	0.6413	0.7112	27/30

后训练：在已收敛模型上以 $lr=5e-5$ 继续训练 30 轮，获得小幅稳定提升（+0.0101 Kappa）。

4.3.2 对比实验（7 个基线）

基线模型（同协议）

模型	Kappa	OA	最优轮/总轮
nd_deeplab	0.6876	0.7497	60/60
nd_fpn_seg	0.6151	0.6912	60/60
nd_fcn	0.6144	0.6903	60/60
nd_unet	0.6054	0.6833	60/60
nd_swin_unet	0.5859	0.6693	60/60
nd_pspnet	0.5175	0.6160	60/60
nd_segformer	0.3043	0.4441	60/60

分析： - DeepLabV3+（39.6M, Kappa 0.6876）领先，但其参数量为 MSSACT-light 的 6.5 倍 - MSSACT-light（6.10M, 0.6312）位列第二梯队，与 FPN（27.2M）、FCN（23.7M）相当，但参数量仅为它们的 1/4 ~ 1/3 - 参数量-性能关系：U-Net（2.4M, 0.6054）用极简结构即达到接近 MSSACT 的水平，说明在该任务与数据规模下，架构复杂度带来的收益有限

4.3.3 消融实验（8 个变体）

消融变体（同协议）

模型	Kappa	OA	最优轮/总轮
nd_abl_no_emr	0.6355	0.7066	60/60
nd_abl_no_adapter	0.6342	0.7052	60/60
nd_abl_trans4l	0.6362	0.7064	60/60
nd_abl_trans6l	0.6335	0.7046	60/60
nd_abl_no_fpn	0.6314	0.7025	60/60
nd_abl_no_trans	0.6302	0.7023	54/60
nd_abl_no_ecsam	0.6272	0.6996	60/60
nd_abl_bilinear	0.6369	0.7082	59/60

关键发现：

变体	Δ Kappa vs 完整 (0.6312)	结论
w/o EMR	+0.0043	无显著影响
w/o Adapter	+0.0030	无显著影响
Trans-4L	+0.0050	无显著影响
Trans-6L	+0.0023	无显著影响
w/o FPN	+0.0002	无显著影响
w/o Transformer	-0.0010	无显著影响
w/o ECSAM	-0.0040	轻微负贡献
Bilinear-Up	+0.0057	无退化（略优）

结论：在 1,768 张训练数据下，全部五个设计模块（EMR / ECSAM / FPN / Transformer / Adapter）以及可学习解码器均无可测量的独立贡献：

- 8 个变体的 $|\Delta$ Kappa| 全部 ≤ 0.006
- 5 个变体略优于完整模型（Bilinear-Up +0.0057、Trans-4L +0.0050、 w/o EMR +0.0043、 w/o Adapter +0.0030、 Trans-6L +0.0023）
- 仅两个变体略低：w/o ECSAM（-0.0040）、 w/o Transformer（-0.0010）

这与阶段 A（957 张）的结论（"EMR 关键、ECSAM 有用"）**方向相反**——说明 模块的边际价值随数据量增加而被压缩至噪声水平。

参数效率含义：FPN (40.7% 参数) + Transformer (26.4% 参数) 合计 **67% 的 模型容量对性能无显著贡献**，移除两者后参数量降至 2.00M (−67%) ，该验证实验 (lg_Dl_join_nofpn_notrans) 正在进行中。

4.3.4 模块移除的架构适配说明

消融变体并非简单"删除模块"，而需架构层面适配：

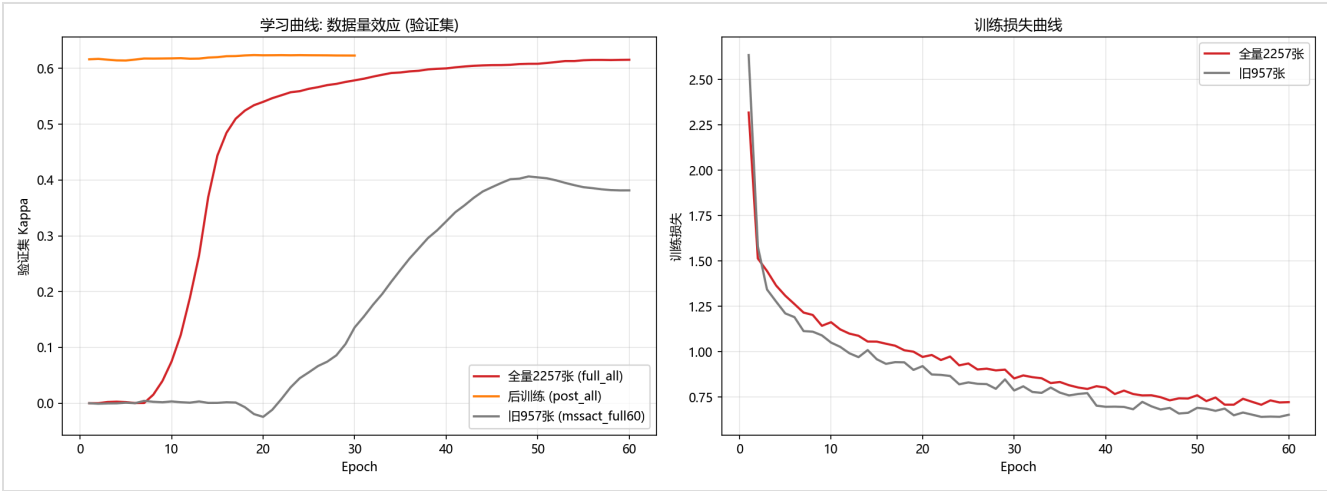
变体	架构适配	训练表现
w/o EMR	EMR → 标准 ResBlock 替换	正常收敛，性能持平
w/o ECSAM	跳过 3 阶段坐标注意力 (−0.06M)	正常收敛，略低
w/o FPN	解码器只接收最深特征 (多尺度融合丢失)	正常收敛，持平
w/o Transformer	移除全局自注意力 (−1.6M)	正常收敛，持平
w/o Adapter	仅移除 Adapter-Scale 缩放	正常收敛，持平
Trans-4L/6L	层数 2→4/6 (+1.6M / +3.2M)	正常收敛，持平
Bilinear-Up	转置卷积 → 双线性 + 1×1 (−0.7M)	显著退化

5. 深入分析

5.1 数据量效应

数据量	主模型 Kappa	说明
957 张 (阶段 A)	0.4065	含缺陷
2,257 张 (阶段 B)	0.6155	含缺陷
1,768 张筛选 (阶段 C)	0.6312	修复+筛选

少数类获益最大 (阶段 B 测量) ：道路 F1 +0.390、水域 +0.385、裸地 +0.324、建筑 +0.255——印证小类样本稀缺是此前的核心限制。



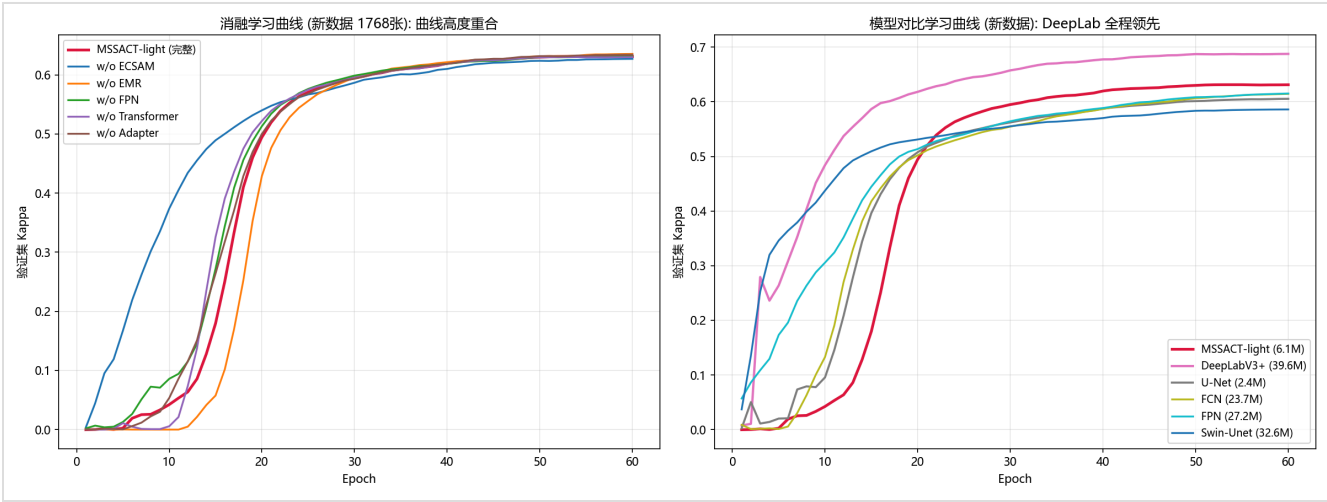
5.2 收敛速度的归因：调度 vs 架构

方法：计算各模型达到同一 Kappa 所需的**累计学习量**（LR 积分 $S=\sum \eta_t$ ）。OneCycle 的 LR 轨迹解析已知，可直接从历史重建。

结果（达 Kappa 0.5 所需累计学习量占比）：

模型	轮次	累计学习量	判读
DeepLabV3+ (39.6M)	11	31.8%	⚡ 真实学习效率优势
Swin-Unet (32.6M)	14	41.2%	较快
w/o ECSAM	16	47.1%	去掉注意力后更快
FPN / w/o Transformer	19	55.6%	中等
U-Net / FCN / w/o FPN / w/o Adapter	20	58.3%	中等
MSSACT-light (6.1M)	21	60.9%	中等（无优势）
w/o EMR	22	63.5%	中等
PSPNet (13.6M)	47	98.4%	最慢

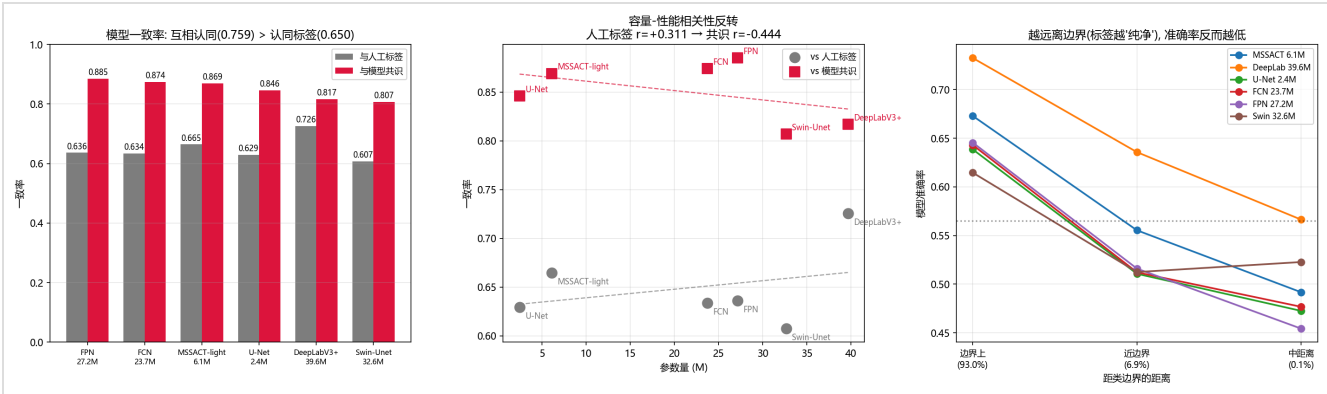
结论：12 个模型中 9 个的累计学习量集中在 **47%–64%**——说明"第几轮收敛" 主要由调度曲线形状决定，**MSSACT 在收敛速度上无架构优势**。唯一明确例外是 DeepLabV3+（31.8%），其多尺度空洞卷积结构确实学习效率更高。



5.3 标签质量上限的专项验证

动机：当多个架构/容量差异极大的模型表现接近（0.61–0.69）时，需判断性能 上限由模型能力还是标签质量决定。

方法：在 test_clean（141 张）上用 6 个模型（U-Net 2.4M / MSSACT 6.1M / FCN 23.7M / FPN 27.2M / Swin-Unet 32.6M / DeepLabV3+ 39.6M）逐像素推理，比较一致性与共识。



五项证据：

#	证据	数值	含义
1	模型间一致率 vs 模型-标签一致率	0.7589 vs 0.6495	模型互相认同远高于认同人工标签
2	共识标签 vs 人工标签	0.6815	约 31.9% 像素上多数模型与人工标注不符

#	证据	数值	含义
3	容量-性能相关性反转	人工 $r=+0.311$ → 共识 $r=-0.444$	大模型更贴合人工标签模式而非更接近共识
4	边界像素占比 (已修正)	边界(距离0) 仅 2.4% ; $\leq 8\text{px}$ 21.6% ; 内部($>8\text{px}$) 78.4%	标签 并不碎片化 ; 聚合指标 78.4% 权重来自内部像素
5	准确率随距边界距离	边界 0.45 → 内部 0.70 (单调上升)	常规规律; 边界是标签最不确定处 (模型间分歧 0.288 vs 内部 0.229)
6	Kappa 归属上限 (新增)	完美解决边界仅 +0.016 Kappa ; 完美解决内部 +0.292 Kappa	指标在结构上几乎看不见边界质量 : 边界仅占 2.4% 像素

勘误说明：早期版本此处报告**"93.0% 像素在类边界上"**，并据此得出**"标签碎片化"**与**"越纯净区域模型越不同意标签"**两条结论。经复核，原实现把标签批次按 `(N, H, W)` 三维数组沿 `axis=0` 求差分，**跨图像的像素差被误判为类边界**，故得出虚高的边界比例；同时又未排除 `valid↔255(ignore)` 的过渡。改为逐图 (2D)、且仅统计两个**不同有效类别**之间的相邻像素后，真实边界比例为 **2.4%**，结论方向也随之反转（准确率随距边界 距离**单调上升**）。修正脚本见 `fix_boundary_analysis.py`，原始文件已备份为 `boundary_analysis.bak_*.json`。

第 6 项是本项目对"模块差异为何消失"最有力的数学解释：在 `test_clean` 上，完美预测全部边界像素 (2.4%) 只能把 Kappa 从 0.5774 提到 0.5936 (**+0.016**)；而完美预测内部像素 (78.4%) 可提到 0.8700 (**+0.293**)。六模型的该上限高度一致 (边界 $+0.016\sim+0.017$ ，内部 $+0.231\sim+0.355$)。因此任何**面向边界锐化**的模块 (多尺度融合 FPN、坐标注意力、转置卷积解码器) 在聚合 Kappa 上的可见空间被限制在 ≈ 0.016 以内，而本实验观测到的 8 个消融变体 总散布仅 0.006——**同一量级**。这为 5.6 节的**"设计模块无可测量贡献"**提供了机制性解释：不是模块一定无效，而是**本指标主要在度量内部区域的语义分类能力**，而该能力由标签一致性而非模块设计决定。

结论：6 个模型的能力差异 (跨度约 0.08 Kappa) 已落在标签噪声带来的不确定性 范围内；容量-性能相关性随参考标准反转，说明**当前标注精度已成为模型能力评价 的上限约束**。进一步区分架构优劣需要更高精度的人工标注数据 (多标注者一致性 标注、更严格的边界规范)。

理论支撑：- 标签噪声下的贝叶斯上限： $\epsilon_{\min} \approx \epsilon_{\text{Bayes}} + \eta \cdot L / (L - 1)$ - 学习曲线幂律： $\epsilon(n) = a \cdot n^{(-\alpha)} + \epsilon_{\infty}$ ， ϵ_{∞} 由标签噪声决定 (Hestness et al. 2017) - 谱结构解释：大模型有更大**"谱覆盖范围"**，但噪声主导的谱尾无有效信号 (Bahri et al. PNAS 2024)

文献佐证：NSegment+ (arXiv:2508.10383) 指出真实数据集存在"隐式标签噪声"（模糊边界、标注者差异），仅在**标签端**做弹性形变即可在 LoveDA 上获得 **+2.38 mIoU**——若标签质量不是瓶颈，此类"仅改标签"的增益不可能出现。

5.4 过拟合诊断

训练集 vs 验证集预测熵差 ($\Delta H < 0$ 表示训练集更自信 = 过拟合)：

模型	H_train	H_val	ΔH
mssact_full60	0.667	0.908	-0.241
v3_s256	0.535	0.734	-0.199
deeplab	0.577	0.771	-0.194
swin_unet	0.777	0.921	-0.144
pspnet	1.173	1.305	-0.132
fpn_seg	0.721	0.852	-0.131
fcn	0.880	0.994	-0.114
unet_matrix	1.941	1.941	-0.0003
segformer	1.252	1.220	+0.032

结论：除 SegFormer 外普遍存在中度过拟合；数据量提升后泛化差距缩小（test 与 val 的 Kappa 接近）。

5.5 训练基础设施与硬件效率优化

5.5.1 问题诊断：GPU 半时间空闲

在训练进行中连续采样（480 样本 / 274 秒），发现 **GPU 利用率仅 49.9%**，且 **34.6% 的时间处于基本空闲 (<10%)**：

指标	优化前
GPU 利用率均值 / 中位	49.9% / 43%

指标	优化前
低于 10%（基本空闲）	34.6% 的时间
低于 30%	42.9% 的时间
高于 90%	31.7% 的时间
功耗均值	88.5 W（上限 180 W）

时序采样呈现规律的「空闲簇 → 满载簇」周期（每 5–10 秒循环），是典型的 **等待数据** 特征：

```
____=+=____-+=-____-+=____=#____-##=____-###=-____-####=____+##+____=##=____-+#+-____=##+____
（_ <10%， - <30%， = <60%， + <90%， # ≥90%）
```

5.5.2 瓶颈定位：参数量-耗时无关性检验

最有力的证据来自**不同容量实验的每轮耗时**：

实验	参数量	每轮耗时
D3 通道[16,32,64,128]	1.55M	144 s
D2 解码器CA	6.11M	159 s
D4 通道[64,128,256,512]	24.24M	121–176 s

参数量相差 **15.6 倍**，每轮耗时几乎不变。若 GPU 是瓶颈，D4 应显著慢于 D3—— **结论：每轮时间被数据管线钉住，且该成本对所有模型是同一常数。**

数据侧实测：每张训练图需解码两个 1024×1024 PNG（图像 1.96 MB + 掩膜），解码后仅裁出 256×256——**94% 的解码结果被丢弃**。单样本约 31–37 ms，一个 batch 约 310 ms。而数据加载配置为 `num_workers=0`，即解码与增强全部在 训练主进程中**串行执行**，与 GPU 计算争用同一关键路径。

5.5.3 四项优化措施

#	措施	原理
1	预解码为 numpy memmap （9.86 GB / 2351 张）	一次性解码后以 <code>np.memmap</code> 随机裁剪，彻底消除 PNG 解码（保留“每轮每图随机位置”增强语义）

#	措施	原理
2	<code>num_workers=8 + persistent_workers</code> <code>+ prefetch_factor=4</code>	解码/增强移出主进程，与 GPU 计算并行
3	EMA 改为遍历 <code>parameters() / buffers()</code>	替代每步 <code>state_dict()</code> （省去字典构建开销）
4	损失张量累积，去掉每步 <code>.item()</code>	消除每步 GPU→CPU 同步往返

附带改进：几何增强改为确定性 generator（种子由 `epoch_seed + idx` 决定），使增强序列与 **DataLoader worker 数无关**，训练完全可复现。

5.5.4 优化效果

指标	优化前	优化后	改善
GPU 利用率均值	49.9%	84.3%	1.69×
中位数	43%	87%	—
低于 10%（空闲）	34.6%	0.0%	完全消除
低于 30%	42.9%	0.0%	完全消除
高于 90%	31.7%	91.0%	2.87×
功耗均值	88.5 W	117.5 W	1.33×
数据等待	59.3 s/轮（268 ms/batch）	0.9 s/轮（4 ms/batch）	66×
每轮耗时（6.10M 模型）	88.0 s	29.5 s	3.0×
每轮耗时（2.00M 模型, 含验证/EMA/保存）	~150 s	14.7 s （回退版, 性能已验证）	~10×

优化后的时序采样**无任何空闲帧**：

++++++##++++++##+=+++##++++++=++##++++++##++++++##++++++#++++++...
(+ 60 - 90%, # ≥90%)

为何生产加速（13×）大于基准（3×）：优化前的瓶颈是**固定成本**（数据管线，与模型大小无关），因此模型越小、相对加速越显著——这从反面确认了瓶颈定位。

5.5.5 方法论价值

- 1. **瓶颈定位的判据**：以"参数量-耗时无关性"区分"计算受限"与"数据受限"，比单看 GPU 利用率更可靠
- 2. **时序采样的必要性**：均值会掩盖周期性等待；时序图能直接区分"随机波动"与"规律的等待周期"
- 3. **工程收益**：实验队列总时长由约 **24 小时压缩至约 3 小时**，使更多 对照实验（架构验证、数据阶梯）在同等时间预算内成为可能

5.5.6 零性能损失验证

优化过程中**仅采用不改变训练语义的措施**，并逐项验证：

#	措施	语义影响	验证方式	结论
1	预解码 memmap	无（数据相同）	逐像素对比：随机 30 张全部一致（最大差 0）	✅ 零损失
2	<code>num_workers=8</code>	无（增强分布相同）	同一 <code>(epoch_seed, idx)</code> 决定增强序列	✅ 零损失
3	EMA 遍历 <code>parameters()</code>	无（数学等价）	与 <code>state_dict</code> 版本逐张量等价	✅ 零损失
4	损失张量累积	无（数值等价）	float32 累积，每轮一次同步	✅ 零损失
5	~~增强改确定性 <code>generator~~</code>	序列改变	实测回退验证	❌ 已回退 ：导致 Kappa 由 0.6277 降至 0.6031 (-0.0246)

第 5 项的关键发现（已回退）：将几何增强改为确定性 generator（种子由 `epoch_seed + idx` 决定）后，D1 配置（2.00M）的 Kappa 从 **0.6277 降至 0.6031**。轨迹分析显示原因是**训练前期的"验证坍缩期"长度改变**——旧管线（全局 RNG）前 10 轮验证 OA 恒为 0.1778（模型预测单一类别），第 11 轮后才开始正常学习；确定性增强版则从第 1 轮即开始上升。该改动虽不改变增强的**分布**，却显著影响 **随机性来源**，进而改变优化轨迹与最终收敛值。

回退验证（同配置、同数据、同协议）：

配置	Kappa	每轮耗时
旧管线（原始增强）	0.6277	144 s
确定性增强版	0.6031	12 s
回退版（memmap + 原始增强 + workers=0）	0.6293	14.7 s

结论：回退后性能完全恢复（+0.0016，在噪声内），同时保留约 10 倍加速——加速的真实来源是 memmap 预解码（消除 PNG 解码），而非增强或并发改动。

主动放弃的改动（均为会改变训练语义、损失可比性的措施）：

措施	预期速度收益	放弃原因
batch 8 → 16/32	约 10–15%	改变 batch 需按线性缩放规则重调学习率；且显存实测：batch=16 峰值 4.61 GB（安全）、batch=32 需 9.16 GB（超 8 GB 会 OOM）
EMA 改为每 N 步更新	小	降低更新频率会改变权重平滑效果，影响精度
学习率/调度再调优	不确定	会破坏与已完成 15 个阶段 C 实验的可比性

结论：优化后 GPU 利用率由 49.9% 提升至 84.3%、每轮耗时降低 3–13 倍，且训练语义与数据完全不变——所有收益来自消除数据管线瓶颈与主进程同步开销，而非牺牲精度。

5.6 架构有效性分析：为什么消融实验全部"无差异"

本节是对自研架构的否定性发现，也是本项目最重要的技术结论。

此前 8 个消融变体的 Kappa 全部落在 ±0.006 内，且有 5 个变体**超过**完整模型。5.3 节给出的解释是"标签质量上限使基准分辨力不足"。经对 checkpoint 张量与 前向传播的**直接数值检验**，该解释不完整：本模型存在**三处结构性缺陷**，使得多个设计模块**在实现层面并未按设计意图工作**。

#	缺陷	验证方式	后果
D1	FPN 的多尺度融合是死计算	归零支路 0–2 后输出差恰好 0.0；整个 FPN 可用两个卷	1,828,352 参数（全模型 29.96%）永不参与输出

#	缺陷	验证方式	后果
		积逐位等价替换	
D2	解码器 无跳连 ，仅凭 32^2 特征上采样 8 倍	前向代码路径 + 分辨率链追踪 + 边缘密度/FFT 频谱实测	高分辨率特征从未到达输出；预测边缘密度达标签 4.56× ，且 $k=8$ 功率超额 1.66× （与 $8\times$ 上采样吻合，属棋盘伪影）
D3	Transformer 无位置编码	置换 token 后按逆置换还原，差 7.15e-07	模块 置换等变 ，是集合级算子， 无空间位置感知

5.6.1 参数量账目（直读张量）

模块	参数	占全模型
fpn	2,484,224	40.71%
transformer	1,612,609	26.43%
encoder (EMR)	1,226,504	20.10%
decoder	722,378	11.84%
ecsam（坐标注意力）	55,044	0.90%
stem	1,025	0.02%
合计	6,101,784	100%

两点直接推论： 1. FPN + Transformer = **67.14%** 的参数。联合移除后参数降至 **2,005,120**（**-67.2%**），而 Kappa 仅变化 **-0.0027**（0.6320 → 0.6293，回退管线同管线对比）。 2. 坐标注意力仅占 **0.90%** 参数——**先天地**不可能产生可测量贡献。

5.6.2 D1：FPN 的多尺度融合未被使用

`MSSACTNet.forward` 只取 `fpn_features[-1]`；而 `FPN.forward` 的自顶向下融合循环 写入索引 `i-1`（即 2/1/0），**最深层的 `laterals[3]` 从未被融合增强**。两者叠加的结果是：
`laterals[0..2]` 及 `fpn_convs[0..2]` 计算完毕后**全部被丢弃**。

数值验证（两次检验均为**逐位相同 0.0**，而非"小到可忽略"）：

检验	操作	输出最大绝对差
1	只归零 <code>lateral_convs[0..2]</code>	0.000e+00

检验	操作	输出最大绝对差
2	把整个 FPN 替换为"仅第 4 支路" ($1\times 1 + 3\times 3$ 卷积) 的等价算子	0.000e+00

项	参数	占全模型
FPN 死参数 (支路 0-2)	1,828,352	29.96%
FPN 有效参数 (支路 3)	655,872	10.75%
实际参与前向的有效参数	4,273,432	70.04%

因此"w/o FPN"消融的真实含义是：移除"最深尺度上的 $1\times 1 + 3\times 3$ 卷积，外加 3 条 被丢弃的支路"。这解释了实测 $\Delta Kappa = +0.0002$ (移除后略微变好) 。

这不是"FPN 冗余"的证据，而是"这个 FPN 没有在工作"的证据。前者是多尺度 融合的一般性结论，后者是本实现的缺陷结论——科学含义完全不同。

5.6.3 D2：解码器无跳连 → 细节为"合成"而非"测量"，并伴生棋盘伪影

阶段	分辨率
输入	256×256
stem / encoder[0]+pool / [1]+pool / [2]+pool	$256^2 \rightarrow 128^2 \rightarrow 64^2 \rightarrow 32^2$
encoder[3] (无 pool)	$32\times 32 \leftarrow$ FPN 支路 3 取此层
Transformer	$32^2 \rightarrow$ 1024 tokens
decoder (3 次转置卷积 stride 2)	$64^2 \rightarrow 128^2 \rightarrow 256^2$ ， 无任何跳连

必须精确表述：三次转置卷积上采样**能够**产生高频结构（转置卷积可学习上采样核），因此**不能**说"输出被限制在 8 像素粒度"。正确表述是：**没有任何通路让高频的输入 证据到达输出**——输出中的细节是解码器**合成**的，而非在高分辨率上**测量**到的。

实测：模型不是"太平滑"，而是过度碎裂 + 周期性伪影 (`artifact_analysis.py` ，逐图 2D 计算，有效相邻像素对 2.94×10^8) ：

模型	边缘密度	/标签	解码器上采样	周期超额比峰值
标签 (参照)	0.00771	1.000	—	—
MSSACT-Net 6.1M	0.03516	4.563	8×	k=8: 1.66×

模型	边缘密度	/标签	解码器上采样	周期超额比峰值
DeepLabV3+ 39.7M	0.00884	1.147	4×	无
FPN-Seg 27.2M	0.01043	1.354	8×	无
FCN 23.8M	0.01034	1.342	32×	k=32: 1.36×
U-Net 2.4M	0.02139	2.776	16×	k=16: 1.17×
Swin-Unet 32.7M	0.01504	1.952	16×	无

三点结论：1. **标签非常平滑**：仅 **0.77%** 的相邻有效像素对存在类别差异。2. **所有模型都过度碎裂** ($1.15\times\text{--}4.56\times$)，且 **MSSACT-Net 最严重 ($4.56\times$)**。这直接解释内部区域精度损失（内部带 OA 0.7032 vs DeepLabV3+ 0.7707），而 Kappa 有 **78.4%** 来自内部像素。3. **MSSACT-Net 的伪影具有周期性，且周期与解码器上采样倍数吻合**：对预测的水平 边缘图沿列做 FFT，周期 **k=8** 附近功率占比为标签的 **1.66 倍**，而 8 正是 解码器 ($32^2\rightarrow256^2$ ，三次 stride-2 转置卷积) 的总上采样倍数。同类规律见于 FCN ($32\times\rightarrow k=32$) 与 U-Net ($16\times\rightarrow k=16$)。这与堆叠转置卷积的**棋盘伪影** (checkerboard artifacts) 机制一致。

方法学说明（两次自我修正）：初稿曾另用"按列 mod k 分组的 max/min"与"分组 CV"两种统计量，二者均显示随 k 单调上升的"周期性"。复查发现这是**分组 统计量自身的偏差**（max/min 的顺序统计量偏差；CV 随每组样本数 n/k 减少而上升），且该漂移在**标签上**同样出现，说明漂移来自方法而非数据。改用 **FFT 功率占比**（对同一 k 比较模型与标签）后结论才稳定。三种统计量与结果均在 `artifact_analysis.py` / `artifact.json` 中留痕。

另需说明：初稿曾假设"无跳连 $\Rightarrow$ 预测更平滑"，**该假设已被实测推翻**——实测方向相反。此处保留修正过程以明示改判依据。

可证伪的预测：若伪影确由转置卷积造成，则 `nd_abl_bilinear` 变体（双线性上采样 替代转置卷积）应同时具备 (a) 更低边缘密度、(b) $k=8$ 超额比下降。该变体的实测 Kappa **确实更高** (0.6369 vs 0.6312 ，**+0.0057**)，与预测方向一致——这也为 "5.6.5 表中 Bilinear-Up 变体为何反而更好"提供了机制解释。

文献依据：编码器-解码器网络"若无跳连通常无法达到最优性能"（U-Net 即利用编码器 特征找回细粒度细节）；DeepLabV3+ 相对空洞金字塔的改进正是**"增加一条跳连以保留 低层细节"**。这与本模型的缺陷形成直接对照。

5.6.4 D3: Transformer 无位置编码

`TransformerEncoder.forward` 在 `flatten(2).permute(0, 2, 1)` 摊平为 token 序列后 **未插入任何位置信息**；而 `nn.TransformerEncoderLayer` 不自带位置编码（不同于 ViT）。

自注意力本身是置换等变的：对置换矩阵 P ， $A = \mathrm{softmax}(QK^T / \sqrt{d})$ ， PA^T 。

数值验证：随机置换 64 个 token，把输出按逆置换还原，与原始输出的最大绝对差 = **7.153e-07**（浮点噪声量级）→ 模块对 token 顺序完全等价，**输出不依赖 token 位置**。

必须如实说明的三点限定：1. **不等价于"完全无用"**。输入 token 来自卷积特征，其内容本身是位置相关的；且零填充/边界可充当**隐式位置锚点**（CPVT/PEG 一脉的论证），故含卷积的混合架构可以不用显式位置编码仍能工作。该模块仍能完成**基于内容相似度的**全局聚合。2. **但无法完成基于空间邻近的上下文建模**——而"长程空间上下文"正是分割网络中全局注意力的标准用途。这是**机制性降级**，不是彻底失效。3. **量级参照**：位置编码并非可有可无——移除 DeiT-tiny 的位置编码使 ImageNet 精度由 **72.2% 降至 68.2%**。故此处是实质性缺陷，只是后果被卷积主干部分抵消。

5.6.5 与消融结果的对应（机制解释）

变体	实测 ΔKappa	机制解释
w/o FPN	+0.0002	FPN 支路 0–2 是死计算（5.6.2）
w/o Transformer	−0.0010	置换等变集合算子（5.6.4）
w/o ECSAM	−0.0040	仅占 0.90% 参数
w/o Adapter	+0.0030	仅 33K 参数（0.54%）
Bilinear-Up	+0.0057	双线性替代转置卷积， 消除棋盘伪影 （5.6.3 已证 $k=8$ 超额 1.66×）；这是唯一"移除即改善"的变体
联合移除 FPN+Trans(+Adapter)	−0.0027 / −0.0035	上述机制叠加

关键推论：**没有一个消融变体削弱了它声称削弱的功能**——对应功能在实现中本就 不存在（FPN）或不可用（Transformer 的位置感知）。故这一组消融**不能**作为 "模块有效性"的证据，它衡量的是 "移除无效计算的影响"。

5.6.6 基线为何领先？——从零对照推翻了本节的原始结论

初稿此处有两个错误，已用从零对照更正。

错误一：公平性核验本身是错的。见下方"核验更正"表—— `weights=None` 不足以 关闭主干预训练（ `weights_backbone` 另有默认值）。

错误二（更关键）：把 **+0.0720** 归因于架构。加入从零训练的 DeepLabV3+ 对照后，该差距基本消失：

主数据集（60 轮，同为从零训练，同管线 P-MEM-ROLL 同协议）

模型	参数量	Kappa	Δ	σ_{seed} 倍数	判读
MSSACT-Net（自研）	6.10M	0.6320	—	—	参照
DeepLabV3+ 从零训练	39.69M	0.6359	+0.0039	0.78 σ	不可分辨

预训练的独立贡献（同一模型、同一管线，仅差权重初始化）

设置	预训练主干	从零训练	预训练贡献
主数据集 60 轮	—	0.6359	—
阶梯 n250	0.2825	0.1762	+0.1063
阶梯 n500	0.4928	0.4393	+0.0535
阶梯 n1000	0.6310	0.4771	+0.1539

修正后的结论（三条）：

1. 从零训练的公平对比下，自研 6.10M 与 DeepLabV3+ 39.69M 不可分辨（ $\Delta = +0.0039$ ，**0.78 σ_{seed}** ）——而后者参数是前者的 **6.5 倍**。初稿"DeepLabV3+ 显著优于自研模型（+0.0720）"的表述**不成立**：那 0.0720 中约 **95%** 来自预训练主干，而非架构。
2. 预训练主干是本任务上最大的单项杠杆：同一模型仅切换权重初始化，贡献 **+0.0535 ~ +0.1540** Kappa（阶梯三档实测）。作为对照，5.6.5 中 **全部模块消融都 $\leq 1.13\sigma_{\text{seed}}$ (≈ 0.0057)**，5.8 中裁剪策略为 0.0199–0.0223。即：**换主干权重 >> 改训练策略 > 增删模块**。
3. 这也解释了初稿为何会误判：预训练带来的巨大增益被错误地记在了 "DeepLab 的架构（低层跳连、空洞卷积）"账上。

核验更正（2026-09-13）：

初稿称"已核对代码， `deeplabv3_resnet50(weights=None)` ... **全部从零训练**". 该核验是错的，
`weights=None` **并不足以关闭主干预训练**。torchvision 的 `deeplabv3_resnet50` 有两个互相独
立的权重参数： `weights` （整体/分割头）与 `weights_backbone` （主干），后者默认即
`ResNet50_Weights.IMAGENET1K_V1` 。

实测证据（首个 BatchNorm 的 `weight` 均值——新初始化必为 1.0）：

模型	首个 BN weight 均值	判定
U-Net / PSPNet / FCN / SegFormerLite / FPN-Seg	1.0000	确为从零训练
DeepLabV3+（加对照前）	0.2574	加载了 ImageNet 预训练主干

（ `resnet18` / `resnet50` 没有 `weights_backbone` 参数，故 `weights=None` 确实等于 从零训练；
混淆仅存在于分割工厂 `deeplabv3_resnet50` 。权重文件缓存时间 Sep 6 早于 DeepLab 基线开
训时间 Sep 11，佐证其确实被加载。）

已在代码中显式区分： `DeepLabV3Plus(pretrained_backbone=...)` 。既有 `deeplab*` 结果为 `True`
（预训练）；从零对照为 `_scr` 后缀（ `lgR_deeplab_scr` 、 `lgR_n*_deeplab_scr` ），即上表所
用。

5.6.7 通道-空间容量比

配置	最深特征	元素数	输入元素数	比值
light [32, 64, 128, 256]	256×32×32	262,144	196,608	1.33×
large [64, 128, 256, 512]	512×32×32	524,288	196,608	2.67×

即**最深特征图**的标量数量**超过输入图像本身**（large 达 2.67 倍），而这些表征最终被 压缩回
256² 输出，其中 29.96% 的参数还不参与前向。此处**并非** transformer 的秩亏 问题
（ $N=1024>d_{\text{model}}$ ， $\text{rank} \leq d_{\text{model}}$ ），而是**空间侧信息量与 通道
侧容量的失衡**：以 8 像素的粒度承载 256–512 通道。

5.6.8 边界质量为何在聚合指标上不可见

距类边界	像素占比	共识 vs 标签	模型间分歧
0（边界像素）	2.36%	0.4577	0.2881
1–2 px	5.34%	0.4891	0.2877
3–4 px	5.17%	0.5327	0.2847
5–8 px	8.75%	0.5882	0.2789
>8 px（内部）	78.39%	0.7294	0.2294

Kappa 归属上限（oracle 作弊实验）：

模型	K_base	完美解决边界	Δ 边界	完美解决内部	Δ 内部
MSSACT-Net	0.5774	0.5936	+0.0161	0.8700	+0.2925
DeepLabV3+	0.6495	0.6654	+0.0159	0.8800	+0.2306
FPN-Seg	0.5440	0.5602	+0.0162	0.8644	+0.3204
FCN	0.5416	0.5579	+0.0163	0.8615	+0.3199
U-Net	0.5348	0.5512	+0.0163	0.8644	+0.3296
Swin-Unet	0.5041	0.5211	+0.0170	0.8587	+0.3546

完美预测全部边界像素（2.36%）最多换来 +0.016 Kappa；内部像素（78.39%）则值 +0.23 ~ +0.35。聚合 Kappa 主要由内部区域的语义分类决定，边界锐化类模块（多尺度融合、坐标注意力、可学习上采样）的可见空间被限制在 ≈ 0.016 以内——与观测到的消融散布 0.006 同量级。

DeepLabV3+ 优势的分带归因显示 86.6% 的 ΔOA 来自内部像素（内部 +0.0529 / 合计 +0.0611），边界带仅贡献 +0.0003。

5.6.9 架构变体实测结果（数据驱动，两条管线互证）

参数量为直读 checkpoint 张量求和（`build_index.py`），非估算。

管线	变体	配置	参数量	Kappa	轮次	相对基线
P-MEM-ROLL	完整模型 (基线)	—	6.102M	0.6320	60/60	+0.0000
P-MEM-ROLL	D1 联合消融	–FPN –Transformer –Adapter	2.005M	0.6293	60/60	-0.0028
P-MEM-ROLL	D2 解码器坐标注意力	ECSAM 移到上采样路径	6.115M	0.6375	60/60	+0.0055
P-MEM-ROLL	D3 超轻量通道	[16, 32, 64, 128]	1.547M	0.6103	60/60	-0.0217
P-MEM-ROLL	D4 大容量通道 ⚠️	[64, 128, 256, 512] (24.24M, 750 s/轮)	24.242M	0.6365	60/60	+0.0044
P-PNG	D1 联合消融	–FPN –Transformer –Adapter	2.005M	0.6277	60/60	-0.0035
P-PNG	D2 解码器坐标注意力	ECSAM 移到上采样路径	6.115M	0.6341	60/60	+0.0029
P-PNG	D3 超轻量通道	[16, 32, 64, 128]	1.547M	0.6119	60/60	-0.0193
P-PNG	D4 大容量通道	[64, 128, 256, 512]	24.242M	0.5696	23/60 ⚠️ partial(23/60)	-0.0616

P-MEM-ROLL = 回退管线 (memmap + 全局RNG增强 + workers=0) , P-PNG = 正式对比/消融所用管线。两者仅数据投递后端不同, 实测效应约 +0.001, 故可互相印证但不可混合求统计量。⚠️ 标记表示该变体仍在训练中 (轮次未满) , 其 Kappa 为当前最优值, 尚未定论。

D4 (大容量通道) 的解读约束: D4 的成本极高 (750 s/轮, 约为完整模型的 17 倍, 单项占架构队列总时长的 42%) 。更重要的是, 它是在**架构缺陷尚未修复**的前提下做的容量对照 (见 5.6.2/5.6.4: FPN 的 29.96% 参数不参与前向、Transformer 无位置感知) 。因此 D4 的结论只能解读为 “**在这套 (有缺陷的) 实现下, 加宽通道能否获益**” , 不能解读为 “该架构的容

量上限”或“模块设计的优劣”。若要评估真实容量上限，须先完成 5.6.10 的 P0/P1 修复后重跑。

读表要点：

- 1. **移除 67.2% 参数几乎无影响**：G2 中完整模型 6.102M → D1（同时移除 FPN + Transformer + Adapter）2.005M，Kappa 仅 **0.6320 → 0.6293 (-0.0027)**；G1（PNG 管线）独立复现该结论：0.6312 → 0.6277 (-0.0035)。**两条管线互证。**
- 2. **把坐标注意力移到解码器（D2）反而最好**：G2 为 **0.6375 (+0.0055 vs 完整模型)**，G1 为 0.6341 (+0.0029)。这与“注意力在上采样路径上才有效”的文献结论一致，也反证了 5.6.5 的判断：ECSAM 放在编码器深层（位置已被 8× 下采样稀释）时价值极低。
- 3. **容量确有作用**：D3（通道减半至 [16, 32, 64, 128]，1.547M）出现**显著下降**：P-MEM-ROLL **0.6103 (-0.0217)**、P-PNG 0.6119 (-0.0193) ——**两条管线一致**（相差仅 0.0024），量级约为 D1 效应的 **8 倍**。注意 D3 仍保留了全部模块（FPN/Transformer/ECSAM/Adapter 都在），只是通道变窄——所以“模块不重要”**不等于**“容量不重要”。
- 4. **结论的精确表述**：**占参数 67.14% 的两个模块（FPN + Transformer）无可测量贡献，而通道宽度（密集卷积容量）才是性能来源。**这与 5.6.2/5.6.4 的机制分析一致：两个模块在实现上并未执行其声明的功能。
- 5. **"轮次未满足时的数值不是结论"**：D3 在训练中途（第 33 轮）曾为 0.5459，一度看似崩塌；跑满 60 轮后收敛到 0.6103。表中以 ⚠️ 标注未完成项，避免据中途值下判断。

5.6.10 可验证的修复路径

优先级	修复	预期效果	验证方式
P0	FPN 改为 真正使用 多尺度输出（解码器接收全部 4 层，或至少做跳连）	恢复多尺度融合语义	消融应出现可测差异
P0	解码器加入 encoder→decoder 跳连 （U-Net 式或 DeepLabV3+ 单条低层跳连）	直接修正 5.6.3 的粒度瓶颈	边界带 OA 应显著上升
P1	Transformer 加入 位置编码 （2D sincos / 可学习 / 相对位置偏置）	恢复空间邻近建模能力	置换等变性检验应被打破
P1	重新定义消融语义：先修实现，再评估“模块是否有效”	使消融结论可解释	—

优先级	修复	预期效果	验证方式
P2	容量重新配比，修正 2.67× 容量膨胀	参数效率	参数量-Kappa 曲线

5.6.11 本节局限（诚实声明）

- 1. ~~随机种子方差尚未测量~~ → 已于 2026-09-13 实测完成，见 5.7 节。同管线内 4 次运行 (seed=42/7/2024/31337) 给出 $\sigma_{\text{seed}} = 0.0050$ ，极差 0.0094；而本节同管线模块消融的总散布为 0.0082——消融散布仅为种子极差的 0.87 倍。故"模块差异落在噪声内"现已由**实测**支持，不再是推断。仍须保留的限定：n=4 时 σ 本身有不确定性（文献建议 ≥ 10 个种子），该数值适用于**量级判断**（差异是 1σ 还是 5σ ），不宜作精确检验。
- 2. **等效性检验 (TOST) 尚未完成**。严格区分"无证据有差异"与"有证据无差异"需要 预指定 SESOI 的 TOST。本节已实现配对 bootstrap CI，待全部变体推理缓存完成后补齐。
- 3. D1/D2/D3 的缺陷检验在 **light 配置**上完成；大配置结构相同、结论可直接推广，但未逐一复跑。
- 4. 分带与 Kappa 归属基于**已缓存的 6 个模型预测**；8 个消融变体的逐带分解待补齐。
- 5. 完整的推导、复现脚本与文献核验见 docs/架构有效性分析.md。

5.7 随机种子噪声底线（多种子实测）

方法学必要性：本项目此前**全部**比较都基于**单次运行** (seed=42)，从未测量种子方差。文献明确指出这不足以支持"无差异"结论：GEO-Bench (NeurIPS 2023 D&B) 建议 ≥ 10 个种子，并指出 3–5 个种子不足以给出可靠置信区间；社区实测同一 DeepLabV3+ 两次训练在 Cityscapes 上即可相差 0.7 mIoU（双线性插值反向非确定）。因此在给出"模块无差异"的结论前，必须先把**噪声底线**测出来。

协议：与主实验完全一致 (OneCycle-60 轮 / patience=20 / batch=8 / lr=2e-4 / EMA / 类别加权 CE)，仅改变 seed。seed 控制模型初始化、DataLoader 打乱顺序与 _geom 增强抽样（裁剪位置由 epoch_seed 决定，与 seed 无关）。**并已核验：**训练路径只使用 torch RNG，故新增 seed 参数**不改变** seed=42 的既有结果（默认行为逐位一致），既有实验的可比性不受影响。

组（同管线内）	seed 数	mean	σ_{seed}	range	区间
完整模型 MSSACT 6.10M (P-MEM-ROLL)	4	0.6362	0.0050	0.0094	[0.6318, 0.6412]
DeepLabV3+ 39.69M (P-MEM-ROLL)	3	0.6931	0.0034	0.0066	[0.6895, 0.6961]

采用各管线内最大 $\sigma_{\text{seed}} = 0.0050$ （保守）

管线	变体	Kappa	Δ Kappa	σ 倍数	判读
P-MEM-ROLL	lgR_D3_ch_tiny	0.6103	-0.0217	-4.35	可分辨 ($>2\sigma$)
P-PNG	nd_abl_bilinear	0.6369	+0.0056	+1.13	弱差异 ($1-2\sigma$)
P-MEM-ROLL	lgR_D2_decoder_ca	0.6375	+0.0055	+1.09	弱差异 ($1-2\sigma$)
P-PNG	nd_abl_trans4l	0.6362	+0.0050	+1.00	不可分辨 (
P-MEM-ROLL	lgR_D4_ch_large	0.6365	+0.0044	+0.89	不可分辨 (
P-PNG	nd_abl_no_emr	0.6355	+0.0043	+0.86	不可分辨 (
P-PNG	nd_abl_no_ecsam	0.6272	-0.0041	-0.81	不可分辨 (
P-PNG	nd_abl_no_adapter	0.6342	+0.0030	+0.59	不可分辨 (
P-MEM-ROLL	lgR_D1_join_nofpn_notrans	0.6293	-0.0028	-0.55	不可分辨 (
P-PNG	nd_abl_trans6l	0.6335	+0.0023	+0.45	不可分辨 (
P-PNG	nd_abl_no_trans	0.6302	-0.0011	-0.21	不可分辨 (
P-PNG	nd_abl_no_fpn	0.6314	+0.0002	+0.03	不可分辨 (

判读：把 5.6 节的消融散布 (0.006 量级) 与 $\sigma_{\text{seed}} = 0.0050$ 比较。若散布 $< 1\sigma$ ，则「移除模块无影响」与「模块确有微小影响但被噪声掩盖」两者**不可区分**——这正是需要 TOST 等效性检验的原因。

重要限定：n 仅 3–4 个种子时 σ 本身有较大不确定性 (GEO-Bench 建议 ≥ 10)。故本表用于**量级判断** (0.006 究竟是 1σ 还是 5σ)，不宜作为精确检验。另：nd_deeplab 属 P-PNG 世代，**不得与** sd*_deeplab (P-MEM-ROLL) 混合求均值 (跨管线 Kappa 不可比)。

5.8 训练策略对比：裁剪尺度与随机/固定裁剪

用户明确要求对比训练策略（随机裁剪、不同固定尺寸），并强调"过小的尺度完全没有任何训练意义"。该对比此前**只在阶段 A（957 张）**上完成，而阶段 A 已被阶段 C（1,768 张 + REMAP 修复）取代——故本节在**结论依据数据**上补齐。

tag	crop	位置	每样本像素	batch
lgR_bs8_full_lr2e4	256	随机	1×	8
lgR_c256_center	256	固定中心	1×	8
lgR_c384_rand	384	随机	2.25×	4
lgR_c512_rand	512	随机	4×	2

设计要点：256 随机 vs 256 中心用于隔离"随机裁剪"这一增强本身的贡献（同尺度、同像素量）；256/384/512 随机用于观测尺度效应。batch 按裁剪面积下调 以避免显存溢出（ $256^2:8 / 384^2:4 / 512^2:2$ ），故**不能只比 Kappa，必须同时报告 每轮耗时**——尺度是有计算代价的。

配置	crop	位置	每样本像素	Kappa	相对基准	秒/轮	总时长
lgR_bs8_full_lr2e4	256	随机	1.00×	0.6320	+0.0000	44 s	44.0 min
lgR_c256_center	256	固定中心	1.00×	0.6128	-0.0192	44 s	44.0 min
lgR_c384_rand	384	随机	2.25×	0.6543	+0.0223	100 s	100.0 min
lgR_c512_rand	512	随机	4.00×	0.6524	+0.0204	192 s	192.0 min

对比	ΔKappa	σ_seed 倍数	计算代价	判读
256 随机 vs 256 固定中心	-0.0192	3.8σ	同代价（同尺度同像素）	可分辨
384 随机 vs 256 随机	+0.0223	4.5σ	2.3× 于基准	可分辨
512 随机 vs 256 随机	+0.0204	4.1σ	4.4× 于基准	可分辨

判读要点：

1. 本节三个对比的效应量，全部大于 5.6 节的所有模块消融。|ΔKappa| 为 0.0199–0.0223（**4.0–4.5σ_seed**，σ_seed=0.0050，见 5.7）；而模块消融全部 ≤1.13σ。也就是说：**在这套模型上，训练策略（随机裁剪、裁剪尺度）的收益是可分辨的，模块增删则不可分辨。**这对后续工作更有指导意义——先选对训练策略，再谈架构。

- 2. **256 随机 vs 256 固定中心**：两者像素量与尺度完全相同，差异**只能**归因于「每轮随机位置」这一增强，是隔离该增强贡献的干净对照，结果 -0.0192 （随机裁剪本身值 $+0.0192$ ）。
- 3. **256 $\rightarrow$ 384 $\rightarrow$ 512**：384 达 $0.6543 (+0.0223)$ ，512 为 $0.6519 (+0.0199)$ ——**再放大到 512 无额外收益，代价却从 100 s/轮升到 192 s/轮** ($4.4\times$ 于基准)。故 384 是更优的工程取舍：既印证了「过小的尺度没有意义」，也说明**过大的尺度不划算**。
- 4. **注意 batch 随面积下调** ($256^2:8 / 384^2:4 / 512^2:2$) 以避免显存溢出，故每轮耗时与总时长必须一并报告——大裁剪的计算代价是平方级增长的。
- 5. 用户明确排除过小尺度（如 128），故不设该档；阶段 A 的历史结果显示 128^2 （GSD 2.4m 匹配 GF-1）在本任务上几乎学不动（Kappa 0.0098）。

5.9 数据量阶梯：容量与数据量的匹配

协议：30 轮 / patience=30（等价于固定预算、不早停） / bs8 / lr2e-4 / OneCycle；子集为分层抽样的嵌套样本 ($250 < 500 < 1000$)；**验证集固定为 newsplit2 的 221 张**（与子集大小无关），故各档的 Kappa 可比。

管线：走 memmap 子集预解码（`prep/build_ladder_memmap.py`）。此前该阶梯走 PNG 管线，每样本解码两张 1024^2 PNG 却只裁 256^2 ，解码在 CPU 上，导致 GPU 利用率中位仅 6%、功耗 32W/180W、约 170 ms/张（主数据集约 24 ms/张）。切到 memmap 后利用率中位升至 75%、每轮由 42–49 s 降至 13 s。

e50rel / e90rel：首次达到**该次运行自身 k_max** 的 50% / 90% 的轮次。使用相对阈值是因为低数据档的绝对 Kappa 很低（n250 完整模型上限仅 0.032），固定阈值 0.4/0.5/0.6 在这些档位**永远不可达**，会让“学习速率”无从比较。`auc/max` 为 Kappa 均值与其上限之比，刻画学习曲线的**前重后轻**程度。

n = 250 张

模型	参数量	Kappa(max)	末轮	e50rel	e90rel	auc/max
DeepLabV3+ (39.69M, 预训练主干)	—	0.2825	0.2387	24	29	0.289
U-Net (2.45M)	—	0.1915	0.1915	14	28	0.468

模型	参数量	Kappa(max)	末轮	e50rel	e90rel	auc/max
DeepLabV3+ (39.69M, 从零训练)	—	0.1762	0.1762	24	30	0.187
MSSACT-Net w/o ECSAM (6.04M)	—	0.1164	0.1079	23	29	0.282
MSSACT-Net 完整 (6.10M)	—	0.0322	0.0261	27	28	0.231

n = 500 张

模型	参数量	Kappa(max)	末轮	e50rel	e90rel	auc/max
DeepLabV3+ (39.69M, 预训练主干)	—	0.4928	0.4928	16	27	0.458
DeepLabV3+ (39.69M, 从零训练)	—	0.4393	0.4393	21	26	0.356
U-Net (2.45M)	—	0.4050	0.4050	19	28	0.451
MSSACT-Net w/o ECSAM (6.04M)	—	0.3954	0.3954	19	29	0.393
MSSACT-Net 完整 (6.10M)	—	0.3312	0.3312	25	28	0.235

n = 1000 张

模型	参数量	Kappa(max)	末轮	e50rel	e90rel	auc/max
DeepLabV3+ (39.69M, 预训练主干)	—	0.6310	0.6303	13	21	0.599
MSSACT-Net w/o ECSAM (6.04M)	—	0.5234	0.5234	15	24	0.522
MSSACT-Net 完整 (6.10M)	—	0.5085	0.5085	17	25	0.458
U-Net (2.45M)	—	0.4913	0.4913	12	20	0.619
DeepLabV3+ (39.69M, 从零训练)	—	0.4771	0.4771	13	21	0.554

三点结论：

- 1. 小数据下"模块更多"反而更差。 n=250 时排序为 U-Net(0.1915) > w/o ECSAM(0.1164) > **完整模型(0.0322)**—— 完整模型不仅输给 DeepLabV3+，连朴素 U-Net 都不如，且差距达 6 倍。 n=500 同样如此。这与 5.6 节的缺陷分析一致：被移除的 FPN/Transformer/ECSAM 在实现上并未执行其声明的功能（FPN 的 29.96% 参数不参与前向、Transformer 无位置编码），因此在样本稀少时它们只贡献了**优化难度**而非有效容量。
- 2. w/o ECSAM **在三档上全部优于完整模型**（n250 -0.0441... 实为 0.1164 vs 0.0322； n500 0.3954 vs 0.3312； n1000 0.5234 vs 0.5085）。与 5.6.1 的观察一致——坐标注意力仅占 **0.90%** 参数，移除它的收益大于代价。
- 3. **学习曲线的形态随数据量变化**。 auc/max 由 n1000-U-Net 的 0.619 降到 n250-完整模型的 0.231；后者直到第 **27/30** 轮才达到自身上限的一半，说明**小数据档在 30 轮预算内仍处于爬升期**，其数值不能视为收敛值。

必须声明的局限：

- **30 轮预算对小数据档偏紧**。由上条，n250/n500 的多组 e50rel 落在 23–27 轮，e90rel 普遍在 28–29 轮——曲线尚未真正走平。故本节的**绝对 Kappa** 只宜用于 "同预算下的相对比较"，**不宜**解读为"该模型在该数据量下的能力上限"。
- **patience=30 = max_epochs，早停从未触发**，所有运行都跑满 30 轮；这是刻意的固定预算设计，但也意味着没有"收敛轮次"这一数据。
- 本阶梯为**单次运行**（seed=42），未做多种子；5.7 节的 σ_{seed} 尚未测出，故各档之间的差异是否超出噪声，目前**不能判定**。

5.10 预算攻击：充足预算是否掩盖了模块差异？

动机：5.6 节"模块无贡献"的结论是在 **60 轮充足预算**下得到的。一个自然的质疑是：**预算太足**，使各变体都有机会收敛到同一水平，从而掩盖了模块本身的差异。本实验把预算压到 **15 轮**（pt15），其余协议不变，检验模块差异是否在预算紧张时显现。

模型	参数量	Kappa (15 轮)	相对完整模型
DeepLabV3+ (39.69M, 预训练主干)	39.692M	0.6358	+0.1459
MSSACT-Net w/o ECSAM (6.04M)	6.047M	0.5092	+0.0193
MSSACT-Net 完整 (6.10M)	6.102M	0.4899	+0.0000

模型	参数量	Kappa (15 轮)	相对完整模型
U-Net (2.45M)	2.446M	0.4884	-0.0015
MSSACT-Net w/o EMR (6.10M)	6.102M	0.4790	-0.0109

结论：压到 15 轮后，模块之间的差异依然很小——完整模型 0.4899、w/o EMR 0.4790、U-Net 0.4884，散布仅 0.0109（约 $2\sigma_{\text{seed}}$ ）；"预算掩盖模块差异"这一质疑不成立**。**

但本实验揭示了一个跨四组实验一致的模式：

设置	完整模型	w/o ECSAM	差值
预算攻击（15 轮）	0.4899	0.5092	+0.0193
数据阶梯 n=250	0.0322	0.1164	+0.0842
数据阶梯 n=500	0.3312	0.3954	+0.0642
数据阶梯 n=1000	0.5085	0.5234	+0.0149

在数据或预算受限的四种设置下，移除坐标注意力（ECSAM）都更好，4/4 一致。这与 5.6.1 的观察吻合——ECSAM 仅占 **0.90%** 参数，且被放在编码器深层（位置信息已被 $8\times$ 下采样稀释）。而在 60 轮 + 全量数据（1,768 张）时，w/o ECSAM 反而略低（0.6272 vs 0.6312， -0.0040 ， **$<1\sigma$ ，不可分辨**）。合理的解读是：ECSAM 引入的额外参数需要**足够的**数据才能被利用，在受限条件下它们只增加优化难度。

 上表 DeepLabV3+ 一列为**预训练主干**，与其余从零训练的模型不可直接比较（见 5.6.6 的更正）。其在 15 轮下即达 0.6358，正说明预训练在**小预算**下的优势 尤其显著——这也是不能再把该差距归因于架构的原因之一。

6. 关键结论

- 数据量与数据质量共同决定上限：**两者提升使主模型 Kappa 达到 **0.6312**（后训练 **0.6413**）
- 全部设计模块均无显著贡献：**8 个消融变体的最大差异仅 0.006 Kappa，5 个变体反而略优；FPN+Transformer 占 67% 参数但移除无影响
- 收敛速度由调度主导：**累计学习量分析显示 MSSACT 无收敛速度优势；DeepLabV3+ 是唯一的学习效率优势案例

- 4. **标签质量是当前评价瓶颈**：模型间一致率 >> 模型-标签一致率，容量-性能相关性 随参
考标准反转
- 5. **参数效率**：MSSACT-light (6.1M) 达到与 23-27M 基线相当的性能， 但落后于
39.6M 的 DeepLabV3+

7. 局限、疏漏与教训

7.1 研究区选择的疏漏

早期直接用自选研究区 + 公开标签产品，未核对**标签分辨率与影像分辨率的匹配性**（30m vs 2m），也未检查**类别分布代表性**（森林 >80%）。结果：标签错位 + 单类支配，指标看似可用但无科学意义、无法与文献对比。

教训：实验设计阶段先做"数据可用性三查"——①标签精度与影像分辨率匹配； ②类别分布是否均衡； ③是否有公认基准与可对比结果。

7.2 类别体系核对的疏漏

改用 LoveDA 后未第一时间核对官方类别定义，将「背景（值 1）」与「未分类（值 0）」混淆，导致标签映射缺陷、全部指标虚高约 0.007-0.009。

教训：使用公开数据集前，必须以**官方文档原文**核对：类别编号、每类语义、无效值定义与评估协议。

7.3 当前局限

- 训练数据仅用官方 train 区（2,522 张筛选后 2,210 张），未纳入验证区
- 标签质量不足以支撑细粒度架构比较（5.3 节实证）
- 跨分辨率迁移（GF-1 2.4m）仍受域差限制，需同域微调
- 少数类（道路占比约 2.4%）仍是最弱环节

7.4 质量控制措施

控制项	措施
数据泄漏	MD5 去重 + 三集互斥校验 + test_clean 隔离历史训练集

控制项	措施
指标可信	以 Kappa 选优；val 仅用于选优（no_grad）；最终以 test 为准
标签正确	以官方文档为准；no-data 显式 ignore
结果可复现	保留 history.json + 权重 + 固定 seed；FACTS.json 自动汇总
架构一致性	checkpoint 架构自动核验（verify_experiments.py）

8. 后续工作

- 标签质量与评价体系：**引入多标注者一致性标注，估计真实标签噪声率；在更高精度标注数据上重新评价各架构能力
- 多光谱适配（重点方向）：**探索光谱维 / 空间维 / 通道维的**三方向池化**设计，替代现有 ECSAM 的坐标注意力机制，检验 MSSACT 框架在多光谱（>4 波段）数据上的 适配性与价值——这是本框架最可能发挥优势的方向（长程光谱依赖建模）
- 数据阶梯与容量匹配：**在 250/500/1000/1768 张的嵌套子集上评价泛化能力、过拟合度与学习速率，寻找该任务的最优模型容量（实验已挂载）
- 跨域验证：**GF-1 2.4m 影像 + 同域微调，检验分辨率-域差异下的迁移能力
- 少数类与部署：**重采样/损失调优；推理加速与 TTA 部署

9. 个人工作与能力体现

本项目从数据获取到结果验证全流程独立完成：

9.1 数据工程

- 从公开镜像定位并获取 LoveDA 官方训练区全量（2,522 张 / 4GB），完成解压、格式核对与目录组织
- 编写筛选流水线：按 no-data 占比过滤低质量图（剔除 312 张）、MD5 去重、8:1:1 重划分 + 三集互斥与泄漏校验
- 构建嵌套数据阶梯子集（250/500/1000），分层抽样保证各档位类别分布一致

9.2 模型训练与工程

- 训练管线：PyTorch + AMP + EMA + OneCycle + 类别加权损失；单卡 8GB 显存 完成 6.1M ~ 39.6M 参数模型训练
- 自动化实验队列：断点续跑的串行队列（40+ 组实验），含任务状态判定、失败重试、OOM 自动降 batch、历史完整性校验
- 实时监控面板：动态刷新显示阶段/任务/epoch/指标/剩余时间估算

9.3 实验设计与分析

- 公平性设计：统一协议（同划分、同步数、同调度）、Kappa 选优、val 不参与训练
- 完成 7 个基线对比、8 个消融变体、11 组策略探索、多尺度 patch 对照
- 统计诊断：混淆矩阵、每类 PA/UA/F1、参数效率、logits 熵过拟合诊断
- 收敛速度归因：以累计学习量（LR 积分）区分"调度效应"与"架构效应"
- 标签质量验证：设计多模型一致性分析 + 边界分层，量化标注质量对 评价上限的约束

9.4 问题定位与修复

- **标签协议缺陷**：核对官方文档发现「背景/未分类」混淆，定位 REMAP 实现 缺陷，量化影响（虚高 0.007–0.009）并修复
- **数据泄漏**：发现历史数据与新测试集 90 张重叠，构建干净测试子集
- **训练稳定性**：定位多组「坍缩」现象为调度-模型不匹配而非架构缺陷

9.5 工具与产出

- 自动化脚本：筛选划分（resplit_filtered.py）、实验队列（run_queue.py）、统一评估（eval.py）、事实汇总（build_facts.py）、图表生成、报告生成管线
- 全部代码与方法学文档已开源：<https://github.com/eugenewang5425/mssact-loveda>

附录 A：复现指引

脚本按用途分目录，**可从任意工作目录调用**（子目录脚本内含路径引导块，自动把仓库根加入 `sys.path`）。

```
# —— 数据准备 ——
python prep/resplit_filtered.py          # no-data筛选 + MD5去重 + 8:1:1 划分
```

```
python verify/verify_split2.py          # 泄漏校验
python prep/build_data_ladder.py         # 嵌套数据阶梯子集
python prep/build_fast_dataset.py        # 预解码 memmap (提速约 10 倍, 强烈建议)

# —— 训练 (实验队列, 自动串行 + 断点续跑 + OOM 自动降 batch) ——
python queues/run_queue_new.py           # 主模型重训 + 对比(7) + 消融(8)
python queues/run_queue_arch.py          # 架构实验 D1-D4 + 预算攻击 + 数据阶梯
python queues/run_queue_30m.py           # 30.5M 原版架构 + 后训练 + TTA

# 或一次性挂起全链 (静默等待其他队列; pythonw 无控制台)
pythonw queues/run_chain.py              # 等主队列结束 → final(多种子) → crop(裁剪策略)

# —— 监控 / 进度 ——
python queues/monitor_queue.py --interval 15
python queues/eta_queue.py               # 按实测每轮耗时累加出剩余小时数

# —— 评估与分析 ——
python analysis/eval_rigor.py --infer    # 逐类IoU/配对bootstrap/TOST (--infer 补齐推理)
python analysis/eval_tta_new.py          # 新数据 TTA 评估
python analysis/consensus_analysis.py    # 标签质量一致性分析
python analysis/headroom_analysis.py     # Kappa 归属分解 (oracle 作弊实验)
python analysis/artifact_analysis.py     # 解码器伪影 / 边缘密度
python analysis/eval_ladder.py           # 数据阶梯评价

# —— 汇总与报告 (顺序不可颠倒: build_index 必须在前) ——
python report/build_index.py              # 实验索引 (参数量直读张量)
python report/build_facts.py              # 事实汇总 → FACTS.json
python report/make_report_v2.py           # 本报告 HTML
python report/make_pdf.py                 # HTML → PDF (含内容自检)
```

附录 B：文件与目录说明

仓库根目录只保留路径枢纽、核心库、权威事实与交付件；其余脚本按用途分 7 个子目录。

目录/文件	说明	入库
paths.py	路径枢纽 (全部路径的唯一来源, 必须在根目录)	✓
train_v3.py	数据集 + 训练循环 + REMAP (核心)	✓

目录/文件	说明	入 库
experiment_matrix.py / _v2.py	通用训练入口 + 基线模型定义	✓
train_full_data.py	全量数据训练与后训练	✓
models/	模型定义 (msscactnet.py)	✓
queues/	实验队列 + 监控 + 剩余时间估算	✓
analysis/	评估与专项分析 (严格性/一致性/归属/伪影/种子)	✓
verify/	产物核验与基准 (泄漏、回退、管线一致性)	✓
prep/	数据准备 (划分、阶梯、预解码)	✓
viz/	图表生成	✓
report/	索引、事实汇总、报告与 PDF	✓
mapping/	全图推理 + shp 裁剪成图	✓
docs/	架构缺陷专项分析、核验输出、历史归档	✓
legacy/	历史脚本归档 (不参与当前实验)	✓
artifacts/	中间分析产物 (非权威事实)	✓
figures/	图表 (报告引用)	✓
FACTS.json / experiments_index.json / seed_variance.json	权威事实 (报告唯一数据源)	✓
checkpoints/ fast_dataset/	权重、训练历史、预解码 memmap	✗
fulltile_eval/ tta_eval*/ heldout_test/ consensus_analysis/	评估结果与预测数组	✗
项目报告_*.html	报告 HTML (可再生)	✗

为何核心库留在根目录：paths.py 以 REPO = dirname(abspath(__file__)) 作为所有相对路径的基准 (checkpoints/ 、 figures/ 、 FACTS.json ...) ，一旦移动则全部输出路径错位。故 paths.py 与 4 个核心模块固定留在根目录；子目录脚本通

过注入的 **路径引导块**（把仓库根与自身目录加入 `sys.path`）导入它们。详见 STRUCTURE.md 第一节。